
7 Error-based Learning

Ever tried. Ever failed. No matter. Try Again. Fail again. Fail better.

—Samuel Beckett

In error-based machine learning, we perform a search for a set of parameters for a parameterized model that minimizes the total error across the predictions made by that model with respect to a set of training instances. The fundamentals section of this chapter introduces the key ideas of a **parameterized model**, measuring **error** and an **error surface**. We then present the standard approach to building error-based predictive models: **multivariable linear regression with gradient descent**. The extensions and variations to this standard approach that we describe are how to handle categorical descriptive features, the use of **logistic regression** to make predictions for categorical target features, fine tuning regression models, techniques for building **non-linear** and **multinomial** models, and **support vector machines**, which take a slightly different approach to using error to build prediction models.

7.1 Big Idea

Anyone who has learned a new sport will have had the, sometimes painful, experience of taking an error-based approach to learning. Take surfing, for example. One of the key skills the novice surfer has to learn is how to successfully *catch a wave*. This involves floating on your surf board until a wave approaches, and then paddling furiously to gain enough momentum for the wave to pick up both you and your board. The position of your body on the board is key to doing this successfully. If you lie too far toward the back of the board, the board will sink and create so much drag that even big waves will pass by, leaving you behind. If you lie too far forward on your board, you will begin to make great progress before the surf board tilts nose down into the water and launches you head over heels into the air. Only when you are positioned at the *sweet spot* in the middle of the board—neither too far forward nor too far back—will you be able to use your paddling efforts to successfully catch a wave.

At their first attempt, new surfers will typically position themselves either too far forward or too far backward on their board when they attempt to catch their first wave, resulting in a bad outcome. The outcome of an attempt to catch a wave is a judgment on how well the surfer is doing, so an attempt constitutes an error function: lying too far back on the board leads to a medium error, lying too far forward on the board leads to a more dramatic error, while successfully catching a wave means really no error at all. Armed with the unsuccessful outcome of their first attempt, surfers usually overcompensate on the second attempt, resulting in the opposite problem. On subsequent attempts, surfers will slowly reduce their error by slightly adjusting their position until they home in on the *sweet spot* at which they can keep their board perfectly balanced to allow a seamless transition to *tickling the face of an awesome toob!*

A family of error-based machine learning algorithms takes the same approach. A parameterized prediction model is initialized with a set of random parameters, and an error function is used to judge how well this initial model performs when making predictions for instances in a training dataset. Based on the value of the error function, the parameters are iteratively adjusted to create a more and more accurate model.

7.2 Fundamentals

In this section we introduce a simple model of linear regression, some metrics for measuring the error of a model, and the concept of an error surface. The discussion in this section, and in the rest of this chapter, assume that you have a basic understanding of differentiation, in particular, what a derivative is, how to calculate a derivative for a continuous function, the chain rule for differentiation, and what a partial derivative is. If you don't understand any of these concepts, see Appendix C^[551] for the necessary introduction.

7.2.1 Simple Linear Regression

Table 7.1^[325] shows a simple dataset recording the rental price (in Euro per month) of Dublin city-center offices (RENTAL PRICE), along with a number of descriptive features that are likely to be related to rental price: the SIZE of the office (in square feet), the FLOOR in the building in which the office space is located, the BROADBAND rate available at the office (in Mb per second), and the ENERGY RATING of the building in which the office space is located (ratings range from A to C, where A is the most efficient). Over the course of this chapter, we look at the ways in which all these descriptive features can be used to train an error-based model to predict office rental prices. Initially, though, we will focus on a simplified version of this task in which just SIZE is used to predict RENTAL PRICE.

Table 7.1

A dataset that includes office rental prices and a number of descriptive features for 10 Dublin city-center offices.

ID	SIZE	FLOOR	BROADBAND RATE	ENERGY RATING	RENTAL PRICE
1	500	4	8	C	320
2	550	7	50	A	380
3	620	9	7	A	400
4	630	5	24	B	390
5	665	8	100	C	385
6	700	4	8	B	410
7	770	10	7	B	480
8	880	12	50	A	600
9	920	14	8	C	570
10	1,000	9	24	B	620

Figure 7.1(a)^[326] shows a scatter plot of the office rentals dataset with RENTAL PRICE on the vertical (or y) axis and SIZE on the horizontal (or x) axis. From this plot, it is clear that there is a strong linear relationship between these two features: as SIZE increases so too does RENTAL PRICES by a similar amount. If we could capture this relationship in a model, we would be able to do two important things. First, we would be able to understand how office size affects office rental price. Second, we would be able to fill in the gaps in the dataset to predict office rental prices for office sizes that we have never

actually seen in the historical data—for example, how much would we expect a 730 square foot office to rent for? Both of these things would be of great use to real estate agents trying to make decisions about the rental prices they should set for new rental properties.

There is a simple, well-known mathematical model that can capture the relationship between two continuous features like those in our dataset. Many readers will remember from school geometry that the **equation of a line** can be written as

$$y = mx + b \quad (7.1)$$

where m is the slope of the line, and b is known as the y -intercept of the line (i.e., the position at which the line meets the vertical axis when the value of x is set to zero). The equation of a line predicts a y value for every x value given the slope and the y -intercept, and we can use this simple model to capture the relationship between two features such as SIZE and RENTAL PRICE. Figure 7.1(b)^[326] shows the same scatter plot as shown in Figure 7.1(a)^[326] with a simple linear model added to capture the relationship between office sizes and office rental prices. This model is

$$\text{RENTAL PRICE} = 6.47 + 0.62 \times \text{SIZE} \quad (7.2)$$

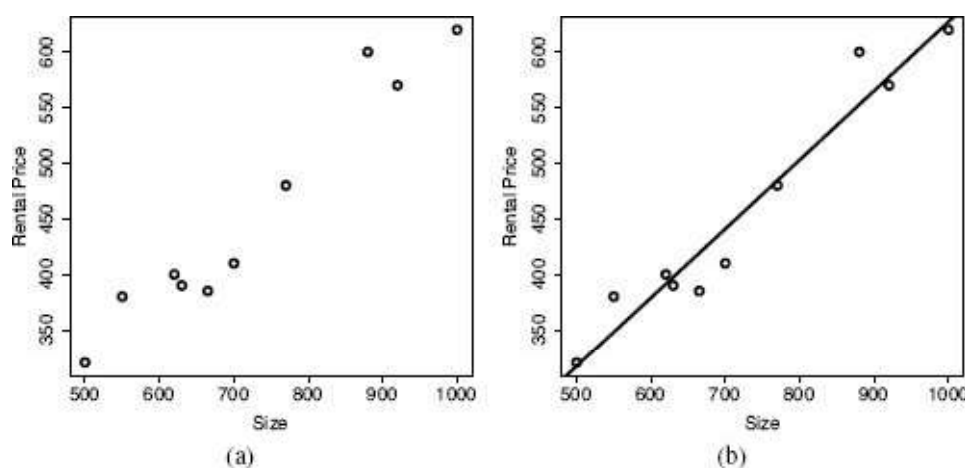


Figure 7.1

(a) A scatter plot of the SIZE and RENTAL PRICE features from the office rentals dataset; (b) the scatter plot from (a) with a linear model relating RENTAL PRICE to SIZE overlaid.

where the slope of the line is 0.62 and the y -intercept is 6.47.

This model tells us that for every increase of a square foot in SIZE, RENTAL PRICE increases by 0.62 Euro. We can also use this model to determine the expected rental price of the 730 square foot office mentioned previously by simply plugging this value for SIZE into the model:

$$\begin{aligned} \text{RENTAL PRICE} &= 6.47 + 0.62 \times 730 \\ &= 459.07 \end{aligned}$$

So, we can expect our 730 square foot office to rent for about 460 Euro per month. This kind of model is known as a **simple linear regression model**. This approach to modeling the relationships between features is extremely common in both machine

learning and statistics.

For consistency with the notation we have used in this book, we can rewrite the simple linear regression model as

$$\mathbb{M}_{\mathbf{w}}(\mathbf{d}) = \mathbf{w}[0] + \mathbf{w}[1] \times \mathbf{d}[1] \quad (7.3)$$

where $\mathbf{w}$ is the vector $\langle \mathbf{w}[0], \mathbf{w}[1] \rangle$, the parameters $\mathbf{w}[0]$ and $\mathbf{w}[1]$ are referred to as weights,¹ $\mathbf{d}$ is an instance defined by a single descriptive feature $\mathbf{d}[1]$, and $\mathbb{M}_{\mathbf{w}}(\mathbf{d})$ is the prediction output by the model for the instance $\mathbf{d}$. The key to using simple linear regression models is determining the optimal values for the weights in the model. The optimal values for the weights are the ones that allow the model to best capture the relationship between the descriptive features and a target feature. A set of weights that capture this relationship well are said to **fit** the training data. In order to find the optimal set of weights, we need some way to measure how well a model defined using a candidate set of weights fits a training dataset. We do this by defining an **error function** to measure the error between the predictions a model makes based on the descriptive features for each instance in the training data and the actual target values for each instance in the training data.

7.2.2 Measuring Error

The model shown in Equation (7.2)^[326] is defined by the weights $\mathbf{w}[0] = 6.47$ and $\mathbf{w}[1] = 0.62$. What tells us that these weights suitably capture the relationship within the training dataset? Figure 7.2(a)^[328] shows a scatter plot of the SIZE and RENTAL PRICE descriptive features from the office rentals dataset and a number of different simple linear regression models that might be used to capture this relationship. In these models the value for $\mathbf{w}[0]$ is kept constant at 6.47 and the values for $\mathbf{w}[1]$ are set to 0.4, 0.5, 0.62, 0.7, and 0.8 from top to bottom. Out of the candidate models shown, the third model from the top (with $\mathbf{w}[1]$ set to 0.62) passes most closely through the actual dataset and is the one that most accurately fits the relationship between office sizes and office rental prices, but how do we measure this formally?

In order to formally measure the fit of a linear regression model with a set of training data, we require an **error function**. An error function captures the error between the predictions made by a model and the actual values in a training dataset.² There are many different kinds of error functions, but for measuring the fit of simple linear regression models, the most commonly used is the **sum of squared errors** error function, or L_2 . To calculate L_2 we use our candidate model $\mathbb{M}_{\mathbf{w}}$ to make a prediction for each member of the training dataset, $\mathcal{D}$, and then calculate the error (or **residual**) between these predictions and the actual target feature values in the training set.

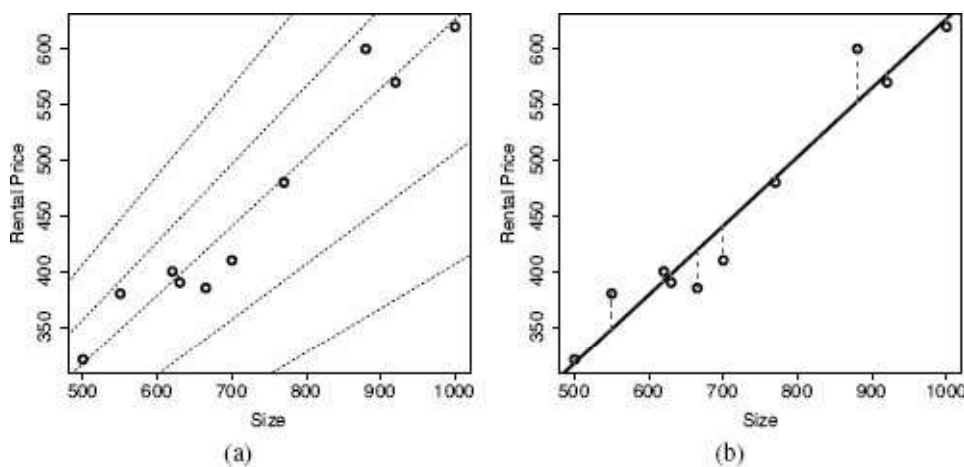


Figure 7.2

(a) A scatter plot of the SIZE and RENTAL PRICE features from the office rentals dataset. A collection of possible simple linear regression models capturing the relationship between these two features are also shown. For all models $\mathbf{w}[0]$ is set to 6.47. From top to bottom, the models use 0.4, 0.5, 0.62, 0.7, and 0.8 respectively for $\mathbf{w}[1]$. (b) A scatter plot of the SIZE and RENTAL PRICE features from the office rentals dataset showing a candidate prediction model (with $\mathbf{w}[0] = 6.47$ and $\mathbf{w}[1] = 0.62$) and the resulting errors.

Figure 7.2(b)^[328] shows the office rentals dataset and the candidate model with $\mathbf{w}[0] = 6.47$ and $\mathbf{w}[1] = 0.62$ and also includes error bars to highlight the differences between the predictions made by the model and the actual RENTAL PRICE values in the training data.

Notice that the model sometimes over estimates the office rental price, and sometimes underestimates the office rental price. This means that some of the errors will be positive and some will be negative. If we were to simply add these together, the positive and negative errors would effectively cancel each other out. This is why, rather than just using the sum of the errors, we use the sum of the *squared* errors because this means all values will be positive.

The sum of squared errors error function, L_2 , is formally defined as

$$L_2(\mathbb{M}_w, \mathcal{D}) = \frac{1}{2} \sum_{i=1}^n (t_i - \mathbb{M}_w(\mathbf{d}_i))^2 \quad (7.4)$$

Table 7.2

Calculating the sum of squared errors for the candidate model (with $\mathbf{w}[0] = 6.47$ and $\mathbf{w}[1] = 0.62$) to make predictions for the office rentals dataset.

ID	SIZE	RENTAL PRICE	Model Prediction	Error	Squared Error
1	500	320	316.47	3.53	12.46
2	550	380	347.47	32.53	1,058.20
3	620	400	390.87	9.13	83.36
4	630	390	397.07	-7.07	49.98
5	665	385	418.77	-33.77	1,140.41
6	700	410	440.47	-30.47	928.42
7	770	480	483.87	-3.87	14.98
8	880	600	552.07	47.93	2,297.28
9	920	570	576.87	-6.87	47.20
10	1,000	620	626.47	-6.47	41.86
Sum					5,674.15
Sum of squared errors (Sum/2)					2,837.08

where the training set is composed of n training instances, each training instance is composed of descriptive features $\mathbf{d}$ and a target feature t , $\mathbb{M}_w(\mathbf{d}_i)$ is the prediction made by a candidate model $\mathbb{M}_w$ for a training instance with descriptive features $\mathbf{d}_i$, and the

candidate model $\mathbb{M}_{\mathbf{w}}$ is defined by the weight vector $\mathbf{w}$. For our simple scenario in which each instance is described with a single descriptive feature, Equation (7.4)^[328] expands to

$$L_2(\mathbb{M}_{\mathbf{w}}, \mathcal{D}) = \frac{1}{2} \sum_{i=1}^n (t_i - (\mathbf{w}[0] + \mathbf{w}[1] \times \mathbf{d}_i[1]))^2 \quad (7.5)$$

Table 7.2^[329] shows the calculation of the sum of squared errors for the candidate model with $\mathbf{w}[0] = 6.47$ and $\mathbf{w}[1] = 0.62$. In this case, the sum of squared errors is equal to 2,837.08.

If we perform the same calculation for the other candidate models shown in Figure 7.2(a)^[328], we find that with $\mathbf{w}[1]$ set to 0.4, 0.5, 0.7, and 0.8, the sums of squared errors are 136,218, 42,712, 20,092, and 90,978 respectively. The fact that the sums of squared errors for these models are larger than for the model with $\mathbf{w}[1]$ set to 0.62 demonstrates that our previous visual intuition that this model most accurately fits the training data was correct.

The sum of squared errors function can be used to measure how well any combination of weights fits the instances in a training dataset. The next section explains how the values of an error function for many different potential models can be combined to form an error surface across which we can search for the optimal weights with the minimum sum of squared errors.³

7.2.3 Error Surfaces

For every possible combination of weights, $w[0]$ and $w[1]$, there is a corresponding sum of squared errors value. We can think about all these error values joined to make a surface defined by the weight combinations, as shown in Figure 7.3(a)^[330]. Here, each pair of weights $w[0]$ and $w[1]$ defines a point on the x - y plane, and the sum of squared errors for the model using these weights determines the height of the error surface above the x - y plane for that pair of weights. The x - y plane is known as a **weight space**, and the surface is known as an **error surface**. The model that best fits the training data is the model corresponding to the lowest point on the error surface.

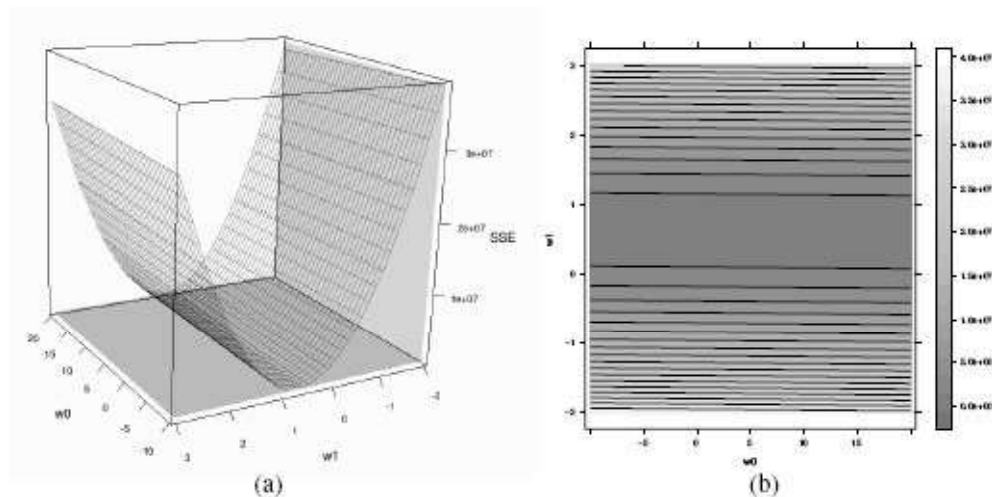


Figure 7.3

(a) A 3D surface plot and (b) a bird's-eye view contour plot of the error surface generated by plotting the sum of squared errors for the office rentals training set for each possible combination of values for $w[0]$ (from the range $[-10, 20]$) and $w[1]$ (from the range $[-2, 3]$).

Although for some simple problems, like that presented in our office rentals dataset, it is possible to try out every reasonable combination of weights and through this **brute-force search** find the best combination, for most real-world problems this is not feasible—the computation required would take far too long. Instead, we need a more efficient way to find the best combination of weights. Fortunately, for prediction problems like that posed by the office rentals dataset, the associated error surfaces have two properties that help us find the optimal combination of weights: they are **convex**, and they have a **global minimum**. By convex we mean that the error surfaces are shaped like a bowl. Having a global minimum means that on an error surface, there is a unique set of optimal weights with the lowest sum of squared errors. The reason that the error surface always has these properties is that its overall shape is determined by the linearity of the model, rather than the properties of the data. If we can find the global minimum of the error surface, we can find the set of weights defining the model that best fits the training dataset. This approach to finding weights is known as **least squares optimization**.

Because we can expect the error surface to be convex and possess a global minimum,

we can find the optimal weights at the point where the **partial derivatives** of the error surface with respect to $\mathbf{w}[0]$ and $\mathbf{w}[1]$ are equal to 0. The partial derivatives of the error surface with respect to $\mathbf{w}[0]$ and $\mathbf{w}[1]$ measure the slope of the error surface at the point $\mathbf{w}[0]$ and $\mathbf{w}[1]$. The point on the error surface at which the partial derivatives with respect to $\mathbf{w}[0]$ and $\mathbf{w}[1]$ are equal to 0 is simply the point at the very bottom of the bowl defined by the error surface—there is no slope at the bottom of the bowl. This point is at the **global minimum** of the error surface and the coordinates of this point define the weights for the prediction model with the lowest sum of squared errors on the dataset. Using Equation (7.5)^[329], we can formally define this point on the error surface as the point at which

$$\frac{\partial}{\partial \mathbf{w}[0]} \frac{1}{2} \sum_{i=1}^n (t_i - (\mathbf{w}[0] + \mathbf{w}[1] \times \mathbf{d}_i[1]))^2 = 0 \quad (7.6)$$

and

$$\frac{\partial}{\partial \mathbf{w}[1]} \frac{1}{2} \sum_{i=1}^n (t_i - (\mathbf{w}[0] + \mathbf{w}[1] \times \mathbf{d}_i[1]))^2 = 0 \quad (7.7)$$

There are a number of different ways to find this point. In this chapter we describe a **guided search** approach known as the **gradient descent** algorithm. This is one of the most important algorithms in machine learning and, as we will see in other chapters, can be used for many different purposes. The next section describes how gradient descent can be used to find the optimal weights for linear regression models that handle multiple descriptive features: **multivariable linear regression** models.

7.3 Standard Approach: Multivariable Linear Regression with Gradient Descent

The most common approach to error-based machine learning for predictive analytics is to use **multivariable linear regression** with **gradient descent** to train a best-fit model for a given training dataset. This section explains how this works. First, we describe how we extend the simple linear regression model described in the previous section to handle multiple descriptive features, and then we describe the gradient descent algorithm.

7.3.1 Multivariable Linear Regression

The simple linear regression model we looked at in Section 7.2.1^[324] handled only a single descriptive feature. Interesting problems in predictive analytics, however, are multivariable⁴ in nature. Fortunately, extending the simple linear regression model to a multivariable linear regression model is straightforward. We can define a multivariable linear regression model as

$$\begin{aligned}\mathbb{M}_{\mathbf{w}}(\mathbf{d}) &= \mathbf{w}[0] + \mathbf{w}[1] \times \mathbf{d}[1] + \cdots + \mathbf{w}[m] \times \mathbf{d}[m] \\ &= \mathbf{w}[0] + \sum_{j=1}^m \mathbf{w}[j] \times \mathbf{d}[j]\end{aligned}\quad (7.8)$$

where $\mathbf{d}$ is a vector of m descriptive features, $\mathbf{d}[1] \dots \mathbf{d}[m]$, and $\mathbf{w}[0] \dots \mathbf{w}[m]$ are $(m + 1)$ weights. We can make Equation (7.8)^[332] look a little neater by inventing a dummy descriptive feature, $\mathbf{d}[0]$, that is always equal to 1. This then gives us

$$\begin{aligned}\mathbb{M}_{\mathbf{w}}(\mathbf{d}) &= \mathbf{w}[0] \times \mathbf{d}[0] + \mathbf{w}[1] \times \mathbf{d}[1] + \cdots + \mathbf{w}[m] \times \mathbf{d}[m] \\ &= \sum_{j=0}^m \mathbf{w}[j] \times \mathbf{d}[j] \\ &= \mathbf{w} \cdot \mathbf{d}\end{aligned}\quad (7.9)$$

where $\mathbf{w} \cdot \mathbf{d}$ is the **dot product** of the vectors $\mathbf{w}$ and $\mathbf{d}$. The dot product of two vectors is the sum of the products of their corresponding elements.

The expansion of the sum of squared errors loss function, L_2 , that we gave in Equation (7.5)^[329] changes slightly to reflect the new regression equation:

$$\begin{aligned}L_2(\mathbb{M}_{\mathbf{w}}, \mathcal{D}) &= \frac{1}{2} \sum_{i=1}^n (t_i - \mathbb{M}_{\mathbf{w}}(\mathbf{d}_i))^2 \\ &= \frac{1}{2} \sum_{i=1}^n (t_i - (\mathbf{w} \cdot \mathbf{d}_i))^2\end{aligned}\quad (7.10)$$

where the training dataset is composed of n training instances $(\mathbf{d}_i, t_i)$, $\mathbb{M}_{\mathbf{w}}(\mathbf{d}_i)$ is the prediction made by a model $\mathbb{M}_{\mathbf{w}}$ for a training instance with descriptive features $\mathbf{d}_i$, and the model $\mathbb{M}_{\mathbf{w}}$ is defined by the weight vector $\mathbf{w}$.

This multivariable model allows us to include all but one of the descriptive features in Table 7.2^[329] in a regression model to predict office rental prices (we will see how to include the categorical ENERGY RATING into the model in Section 7.4.3^[351]). The resulting multivariable regression model equation is

$$\begin{aligned}\text{RENTAL PRICE} &= \mathbf{w}[0] + \mathbf{w}[1] \times \text{SIZE} + \mathbf{w}[2] \times \text{FLOOR} \\ &\quad + \mathbf{w}[3] \times \text{BROADBAND RATE}\end{aligned}$$

We will see in the next section how the best-fit set of weights for this equation are found, but for now we will set $\mathbf{w}[0] = -0.1513$, $\mathbf{w}[1] = 0.6270$, $\mathbf{w}[2] = -0.1781$, and $\mathbf{w}[3] = 0.0714$. This means that the model is rewritten as

$$\text{RENTAL PRICE} = -0.1513 + 0.6270 \times \text{SIZE} - 0.1781 \times \text{FLOOR} \\ + 0.0714 \times \text{BROADBAND RATE}$$

Using this model, we can, for example, predict the expected rental price of a 690 square foot office on the 11th floor of a building with a broadband rate of 50 Mb per second as

$$\text{RENTAL PRICE} = -0.1513 + 0.6270 \times 690 - 0.1781 \times 11 + 0.0714 \times 50 \\ = 434.0896$$

The next section describes how the weights can be determined using the gradient descent algorithm.

7.3.2 Gradient Descent

In Section 7.2.3^[330] we said that the best-fit set of weights for a linear regression model can be found at the global minimum of the error surface defined by the weight space associated with the relevant training dataset. We also mentioned that this global minimum can be found at the point at which the partial derivatives of the error surface, with respect to the weights, are equal to zero. Although it is possible to calculate this point directly for some simpler problems, this approach is not computationally feasible for most interesting predictive analytics problems. The number of instances in the training set and the number of weights for which we need to find values simply make the problem too large. The brute-force search approach that was mentioned in Section 7.2.3^[330] is not feasible either—especially as the number of descriptive features, and subsequently the number of weights, increases.

There is, however, a simple approach to learning weights that we can take based on the facts that, even though they are hard to visualize, the error surfaces that correspond to these high-dimensional weight spaces still have the convex shape seen in Figure 7.3^[330] (albeit in multiple dimensions), and that a single global minimum exists. This approach uses a guided search from a random starting position and is known as **gradient descent**.

To understand how gradient descent works, imagine a hiker unlucky enough to be stranded on the side of a valley on a foggy day. Because of the dense fog, it is not possible for her to see the way to her destination at the bottom of the valley. Instead, it is only possible to see the ground at her feet to within about a three foot radius. It might, at first, seem like all is lost and that it will be impossible for the hiker to find her way down to the bottom of the valley. There is, however, a reliable approach that the hiker can take that will guide her to the bottom (assuming, somewhat ideally, that the valley is *convex* and has a *global minimum*). If the hiker looks at the slope of the ground at her feet, she will notice that in some directions, the ground slopes up, and in other directions, the ground slopes down. If she takes a small step in the direction in which the ground slopes most steeply downward (the direction of the gradient of the mountain), she will be headed toward the bottom of the mountain. If she repeats this process over and over again, she will make steady progress down the mountain until eventually she arrives at the bottom. Gradient descent works in exactly the same way.

Gradient descent starts by selecting a random point within the weight space (i.e., each weight in the multivariable linear regression equation is assigned a random value within some sensible range) and calculating the sum of squared errors associated with this point based on predictions made for each instance in the training set using the randomly selected weights (as shown in Section 7.2.2^[327]). This defines one point on the error surface. Although the error value at this point in the weight space can be calculated, we know very little else about the relative position of this point on the error surface. Just like our imagined mountain climber, the algorithm can use only very localized information. It is possible, however, to determine the slope of the error surface by determining the derivative of the function used to generate it, and then calculating the value of this derivative at the random point selected in the weight space. This means that, again like our

mountain climber, the gradient descent algorithm can use the direction of the slope of the error surface at the current location in the weight space. Taking advantage of this information, the randomly selected weights are adjusted slightly in the direction of the error surface gradient to move to a new position on the error surface. Because the adjustments are made in the direction of the error surface gradient, this new point will be closer to the overall global minimum. This adjustment is repeated over and over until the global minimum on the error surface is reached. Figure 7.4^[336] shows an error surface (defined over just two weights so that we can visualize the error surface) and some examples of the path down this surface that the gradient descent algorithm would take from different random starting positions.⁵

For the simple version of the office rentals example that uses only the SIZE descriptive feature, described in Section 7.2.1^[324], it is easy to visualize how the gradient descent algorithm would move iteratively toward a model that best fits the training data, making small adjustments each time—with each adjustment reducing the error of the model, just as our surfer from Section 7.1^[323] did. Figure 7.5^[337] shows the journey across the error surface that is taken by the gradient descent algorithm when training this model. Figure 7.6^[338] shows a series of snapshots of the candidate models created at steps along this journey toward the best-fit model for this dataset. Notice how the model gets closer and closer to a model that accurately captures the relationship between SIZE and RENTAL PRICE. This is also apparent in the final panel in Figure 7.6^[338], which shows how the sum of squared errors decreases as the model becomes more accurate.

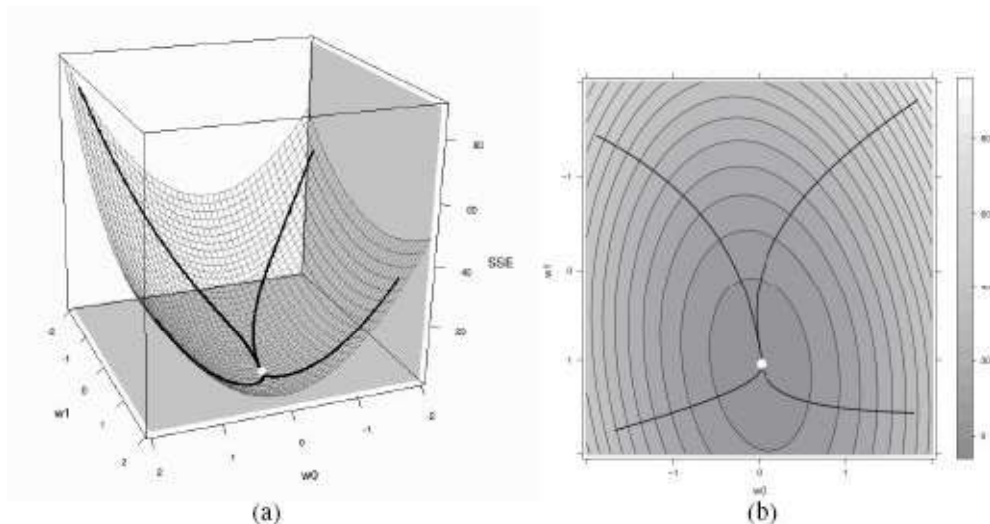


Figure 7.4

(a) A 3D plot of an error surface and (b) a bird's-eye view contour plot of the same error surface. The lines indicate the path that the gradient descent algorithm would take across this error surface from 4 different starting positions to the global minimum—marked as the white dot in the center.

The gradient descent algorithm for training multivariable regression models is formally presented in Algorithm 7.1^[339]. Each weight is iteratively adjusted by a small amount

based on the error in the predictions made by the current candidate model so as to generate subsequently more and more accurate candidate models. Eventually, the algorithm will converge to a point on the error surface where any subsequent changes to weights do not lead to a noticeably better model (within some tolerance). At this point we can expect the algorithm to have found the global minimum of the error surface and, as a result, the most accurate predictive model possible.

The most important part to the gradient descent algorithm is the line on which the weights are updated, Line 4^[339]. Each weight is considered independently, and for each one a small adjustment is made by adding a small value, called a **delta value**, to the current weight, $w[j]$. This adjustment should ensure that the change in the weight leads to a move *downward* on the error surface. The learning rate, α , determines the size of the adjustments made to weights at each iteration of the algorithm and is discussed further in Section 7.3.3^[341].

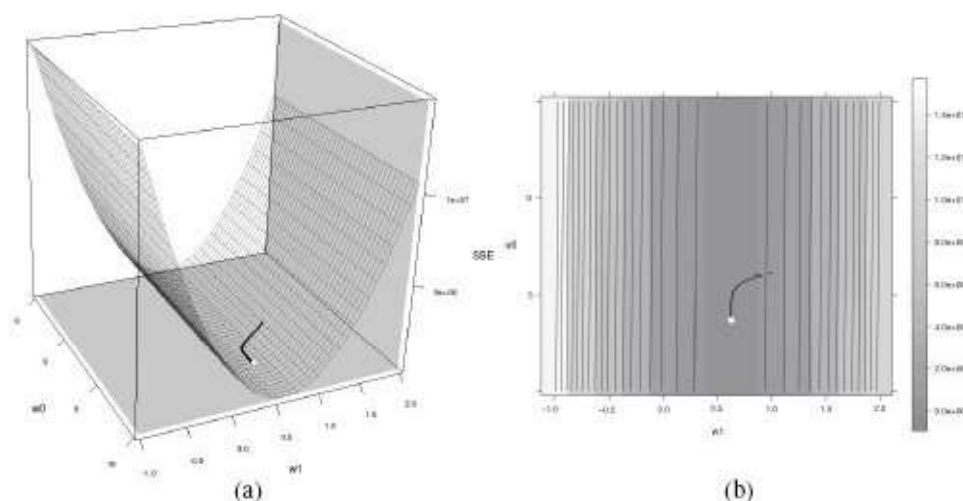


Figure 7.5

(a) A 3D surface plot and (b) a bird's-eye view contour plot of the error surface for the office rentals dataset showing the path that the gradient descent algorithm takes toward the best-fit model.

The remainder of this section focuses on the error delta function, which calculates the delta value that determines the direction (either positive or negative) and the magnitude of the adjustments made to each weight. The direction and magnitude of the adjustment to be made to a weight is determined by the gradient of the error surface at the current position in the weight space. Recalling that the error surface is defined by the error function, L_2 (given in Equation (7.10)^[333]), the gradient at any point on this error surface is given by the value of the partial derivative of the error function with respect to a particular weight at that point. The error delta function invoked on Line 4^[339] of Algorithm 7.1^[339] performs this calculation to determine the delta value by which each weight should be adjusted.

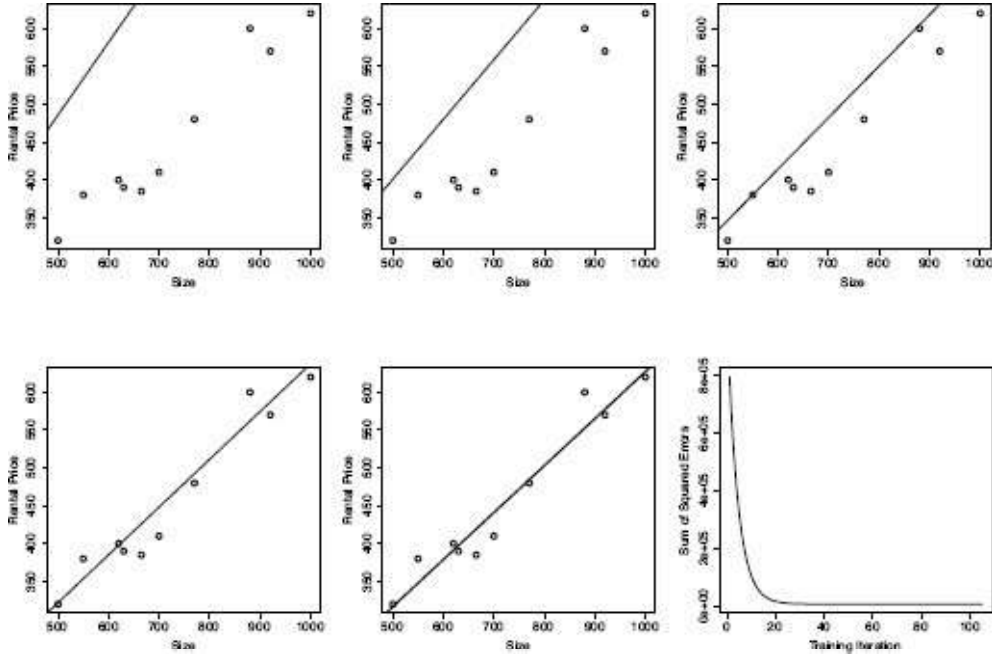


Figure 7.6

A selection of the simple linear regression models developed during the gradient descent process for the office rentals dataset. The bottom-right panel shows the sums of squared errors generated during the gradient descent process.

To understand how to calculate the value of the partial derivative of the error function with respect to a particular weight, let us imagine for a moment that our training dataset, $\mathcal{D}$, contains just one training instance: $(\mathbf{d}, t)$, where $\mathbf{d}$ is a set of descriptive features and t is a target feature. The gradient of the error surface is given as the partial derivative of L_2 with respect to each weight, $\mathbf{w}[j]$:

$$\frac{\partial}{\partial \mathbf{w}[j]} L_2(\mathbb{M}_{\mathbf{w}}, \mathcal{D}) = \frac{\partial}{\partial \mathbf{w}[j]} \left(\frac{1}{2} (t - \mathbb{M}_{\mathbf{w}}(\mathbf{d}))^2 \right) \quad (7.11)$$

$$= (t - \mathbb{M}_{\mathbf{w}}(\mathbf{d})) \times \frac{\partial}{\partial \mathbf{w}[j]} (t - \mathbb{M}_{\mathbf{w}}(\mathbf{d})) \quad (7.12)$$

$$= (t - \mathbb{M}_{\mathbf{w}}(\mathbf{d})) \times \frac{\partial}{\partial \mathbf{w}[j]} (t - (\mathbf{w} \cdot \mathbf{d})) \quad (7.13)$$

$$= (t - \mathbb{M}_{\mathbf{w}}(\mathbf{d})) \times -\mathbf{d}[j] \quad (7.14)$$

Equation (7.12)^[338] is calculated from Equation (7.11)^[338] by applying the differentiation **chain rule**.⁶ To understand the move from Equation (7.13)^[338] to Equation (7.14)^[338] imagine a problem with four descriptive features $\mathbf{d}[1] \dots \mathbf{d}[4]$. Remembering that we always include the dummy feature $\mathbf{d}[0]$ with a value of 1, the dot product $\mathbf{w} \cdot \mathbf{d}$ becomes

$$\begin{aligned} \mathbf{w} \cdot \mathbf{d} &= \mathbf{w}[0] \times \mathbf{d}[0] + \mathbf{w}[1] \times \mathbf{d}[1] + \mathbf{w}[2] \times \mathbf{d}[2] \\ &\quad + \mathbf{w}[3] \times \mathbf{d}[3] + \mathbf{w}[4] \times \mathbf{d}[4] \end{aligned}$$

Algorithm 7.1 The gradient descent algorithm for training multivariable linear regression models.

Require: set of training instances $\mathcal{D}$

Require: a learning rate α that controls how quickly the algorithm converges

Require: a function, *errorDelta*, that determines the direction in which to adjust a given weight, $\mathbf{w}[j]$, so as to move down the slope of an error surface determined by the dataset, $\mathcal{D}$

Require: a convergence criterion that indicates that the algorithm has completed

1: $\mathbf{w} \leftarrow$ random starting point in the weight space

2: **repeat**

3: **for** each $\mathbf{w}[j]$ in $\mathbf{w}$ **do**

4: $\mathbf{w}[j] \leftarrow \mathbf{w}[j] + \alpha \times \text{errorDelta}(\mathcal{D}, \mathbf{w}[j])$

5: **until** convergence occurs

If we take the partial derivative of this with respect to $\mathbf{w}[0]$ all the terms that do not contain $\mathbf{w}[0]$ are treated as constants, so

$$\begin{aligned}\frac{\partial}{\partial \mathbf{w}[0]} \mathbf{w} \cdot \mathbf{d} &= \frac{\partial}{\partial \mathbf{w}[0]} (\mathbf{w}[0] \times \mathbf{d}[0] + \mathbf{w}[1] \times \mathbf{d}[1] + \mathbf{w}[2] \times \mathbf{d}[2] \\ &\quad + \mathbf{w}[3] \times \mathbf{d}[3] + \mathbf{w}[4] \times \mathbf{d}[4]) \\ &= \mathbf{d}[0] + 0 + 0 + 0 + 0 = \mathbf{d}[0]\end{aligned}$$

Similarly, the partial derivative with respect to $\mathbf{w}[4]$ is

$$\begin{aligned}\frac{\partial}{\partial \mathbf{w}[4]} \mathbf{w} \cdot \mathbf{d} &= \frac{\partial}{\partial \mathbf{w}[4]} (\mathbf{w}[0] \times \mathbf{d}[0] + \mathbf{w}[1] \times \mathbf{d}[1] + \mathbf{w}[2] \times \mathbf{d}[2] \\ &\quad + \mathbf{w}[3] \times \mathbf{d}[3] + \mathbf{w}[4] \times \mathbf{d}[4]) \\ &= 0 + 0 + 0 + 0 + \mathbf{d}[4] = \mathbf{d}[4]\end{aligned}$$

So, in the move between Equations (7.13)^[338] and (7.14)^[338] $\frac{\partial}{\partial \mathbf{w}[j]} (t - (\mathbf{w} \cdot \mathbf{d}))$ becomes $-\mathbf{d}[j]$ (remember that in this equation, t is a constant and so becomes zero when differentiated).

Equation (7.14)^[338] calculates the gradient based only on a single training instance. To take into account multiple training instances, we calculate the sum of the squared errors for each training instance (as we did in all our previous examples). So, Equation (7.14)^[338] becomes

$$\frac{\partial}{\partial \mathbf{w}[j]} L_2(\mathbb{M}_{\mathbf{w}}, \mathcal{D}) = \sum_{i=1}^n ((t_i - \mathbb{M}_{\mathbf{w}}(\mathbf{d}_i)) \times -\mathbf{d}_i[j]) \quad (7.15)$$

where $(\mathbf{d}_1, t_1) \dots (\mathbf{d}_n, t_n)$ are n training instances, and $\mathbf{d}_i[j]$ is the j^{th} descriptive feature of training instance $(\mathbf{d}_i, t_i)$. The direction of the gradient calculated using this equation is toward the highest values on the error surface. The error delta function from Line 4^[339] of Algorithm 7.1^[339] should return a small step toward a lower value on the error surface. Therefore we move in the opposite direction of the calculated gradient, and the error delta

function can be written as

$$\begin{aligned} \text{errorDelta}(\mathcal{D}, \mathbf{w}[j]) &= -\frac{\partial}{\partial \mathbf{w}[j]} L_2(\mathbb{M}_{\mathbf{w}}, \mathcal{D}) \\ &= \sum_{i=1}^n ((t_i - \mathbb{M}_{\mathbf{w}}(\mathbf{d}_i)) \times \mathbf{d}_i[j]) \end{aligned} \quad (7.16)$$

Line 4^[339] of Algorithm 7.1^[339] can therefore be rewritten as what is known as the weight update rule for multivariable linear regression with gradient descent:

$$\mathbf{w}[j] \leftarrow \mathbf{w}[j] + \alpha \times \underbrace{\sum_{i=1}^n ((t_i - \mathbb{M}_{\mathbf{w}}(\mathbf{d}_i)) \times \mathbf{d}_i[j])}_{\text{errorDelta}(\mathcal{D}, \mathbf{w}[j])} \quad (7.17)$$

where $\mathbf{w}[j]$ is any weight, α is a constant learning rate, t_i is the expected target feature value for the i^{th} training instance, $\mathbb{M}_{\mathbf{w}}(\mathbf{d}_i)$ is the prediction made for this training instance by the current candidate model defined by the weight vector $\mathbf{w}$, and $\mathbf{d}_i[j]$ is the j^{th} descriptive feature of the i^{th} training instance and corresponds with weight $\mathbf{w}[j]$ in the regression model.

To intuitively understand the weight update rule given in Equation (7.17)^[340], it helps to think in terms of what the weight update rule does to weights based on the error in the predictions made by the current candidate model:

- If the errors show that, in general, predictions made by the candidate model are too high, then $\mathbf{w}[j]$ should be decreased if $\mathbf{d}_i[j]$ is positive and increased if $\mathbf{d}_i[j]$ is negative.
- If the errors show that, in general, predictions made by the candidate model are too low, then $\mathbf{w}[j]$ should be increased if $\mathbf{d}_i[j]$ is positive and decreased if $\mathbf{d}_i[j]$ is negative.

The approach to training multivariable linear regression models described so far is more specifically known as **batch gradient descent**. The word batch is used because only one adjustment is made to each weight at each iteration of the algorithm based on summing the squared error made by the candidate model for each instance in the training dataset.⁷ Batch gradient descent is a straightforward, accurate, and reasonably efficient approach to training multivariable linear regression models and is used widely in practice. The inductive bias encoded in this algorithm includes a preference bias to prefer models that minimize the sum of squared errors function and a restriction bias introduced by the facts that we only consider linear combinations of descriptive features and that we take a single path through the error gradient from a random starting point.

7.3.3 Choosing Learning Rates and Initial Weights

The values chosen for the learning rate and initial weights can have a significant impact on how the gradient descent algorithm proceeds. Unfortunately, there are no theoretical results that help in choosing the optimal values for these parameters. Instead, these algorithm parameters must be chosen using rules of thumb gathered through experience.

The learning rate, α , in the gradient descent algorithm determines the size of the adjustment made to each weight at each step in the process. We can illustrate this using the simplified version of the RENTAL PRICE prediction problem based only on office size (SIZE). A linear regression model for the problem uses only two weights, $w[0]$ and $w[1]$. Figure 7.7^[342] shows how different learning rates—0.002, 0.08, and 0.18—result in very different journeys across the error surface.⁸ The changing sum of squared errors that result from these journeys are also shown.

Figure 7.7(a)^[342] shows the impact of a very small learning rate. Although the gradient descent algorithm will converge to the global minimum eventually, it takes a very long time as tiny changes are made to the weights at each iteration of the algorithm. Figure 7.7(c)^[342] shows the impact of a large learning rate. The large adjustments made to the weights during gradient descent cause it to jump completely from one side of the error surface to the other. Although the algorithm can still converge toward an area of the error surface close to the global minimum, there is a strong chance that the global minimum itself will actually be missed, and the algorithm will simply jump back and forth across it. In fact, if inappropriately large learning rates are used, the jumps from one side of the error surface to the other can actually cause the sum of squared errors to repeatedly increase rather than decrease, leading to a process that will never converge. Figure 7.7(b)^[342] shows that a well-chosen learning rate strikes a good balance, converging quickly but also ensuring that the global minimum is reached. Note that even though the shape of the curve in Figure 7.7(e)^[342] is similar to the shape in Figure 7.7(d)^[342], it takes far fewer iterations to reach the global minimum.

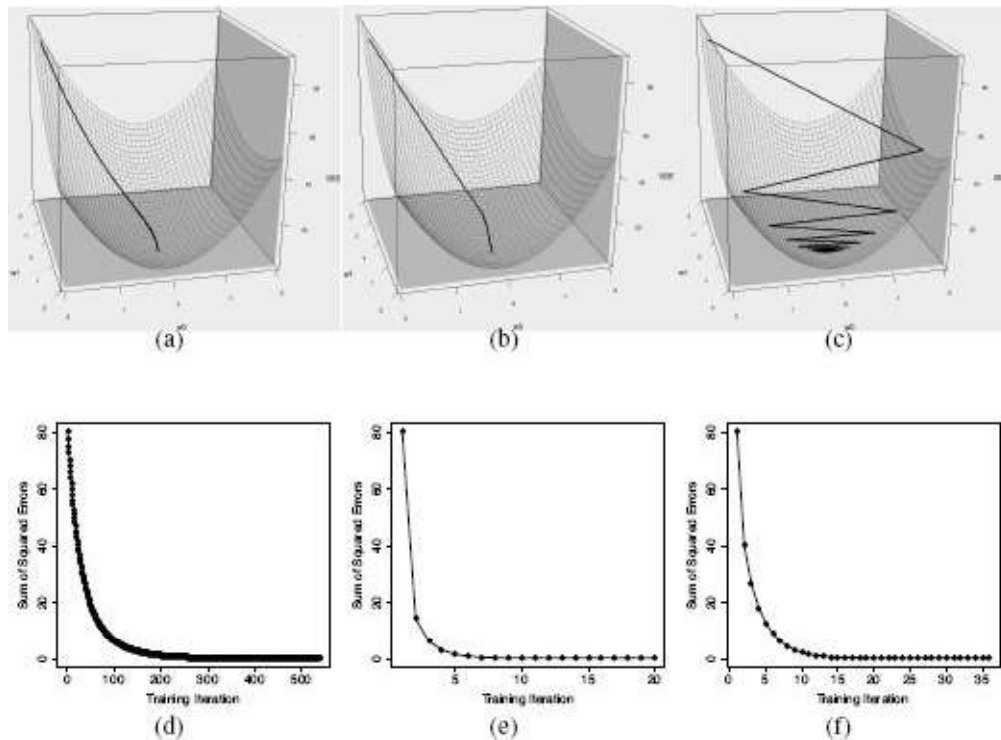


Figure 7.7

Plots of the journeys made across the error surface for the simple office rentals prediction problem for different learning rates: (a) a very small learning rate (0.002); (b) a medium learning rate (0.08); and (c) a very large learning rate (0.18). The changing sum of squared errors for these journeys are also shown.

Unfortunately, choosing learning rates is not a well-defined science. Although there are some algorithmic approaches, most practitioners use rules of thumb and trial and error. A typical range for learning rates is $[0.00001, 10]$, and practitioners will usually try out higher values and observe the resulting learning graph. If the graph looks too much like Figure 7.7(f)^[342], a smaller value will be tested until something approaching Figure 7.7(e)^[342] is found.

When the gradient descent algorithm is used to find optimal weights for linear regression models, the initial weights are chosen randomly from a predefined range that must be specified as an input to the algorithm. The choice of the range from which these initial weights are selected affects how quickly the gradient descent algorithm will converge to a solution. Unfortunately, as is the case with the learning rate, there are no well-established, proven methods for choosing initial weights. Normalization also has a part to play here. It is much easier to select initial weights for normalized feature values than for raw feature values, as the range in which weights for normalized feature values might reasonably fall (particularly for the intercept weight, $w[0]$) is much better defined than the corresponding range when raw feature values are used. The best advice we can give is that, based on empirical evidence, choosing random initial weights uniformly from the range $[-0.2, 0.2]$ tends to work well.

7.3.4 A Worked Example

We are now in a position to build a linear regression model that uses all the continuous descriptive features in the office rentals dataset in Table 7.1^[325] (i.e., all features except for ENERGY RATING). The general structure of the model is

$$\text{RENTAL PRICE} = \mathbf{w}[0] + \mathbf{w}[1] \times \text{SIZE} + \mathbf{w}[2] \times \text{FLOOR} \\ + \mathbf{w}[3] \times \text{BROADBAND RATE}$$

so there are four weights— $\mathbf{w}[0]$, $\mathbf{w}[1]$, $\mathbf{w}[2]$, and $\mathbf{w}[3]$ —for which optimal values must be found. For this example, let's assume that the learning rate, α , is 0.00000002 and the initial weights are chosen from a uniform random distribution in the range $[-0.2, 0.2]$ to be $\mathbf{w}[0] = -0.146$, $\mathbf{w}[1] = 0.185$, $\mathbf{w}[2] = -0.044$, and $\mathbf{w}[3] = 0.119$. Table 7.3^[345] details the important values from the first two iterations of the gradient descent algorithm when applied to this data.⁹

Using the initial weights predictions are made for all the instances in the training dataset, as shown in the Predictions column (column 3) of Table 7.3^[345]. By comparing these predicted values with the actual RENTAL PRICE (column 2), we can compute an error and a squared error term for each training instance, columns 4 and 5 of the table.

To update the weights, we must first calculate the delta value for each weight. This is calculated by summing over all the instances in the training set the prediction error multiplied by the value of the relevant feature for that instance (see Equation (7.16)^[340]). The last four columns on the right of the table list for each instance the product of the prediction error and the feature value. Remember that $\mathbf{d}[0]$ is a dummy descriptive feature, added to match $\mathbf{w}[0]$, with a value of 1 for all training instances. As a result, the values in column 6 are identical to the values in the error column. Focusing on the top cell of column 7, we see the value 113,370.05. This value was calculated by multiplying the prediction error for $\mathbf{d}_1$ (226.74) by the SIZE value for this instance (500). The other cells in these columns are populated with similar calculations. The $\text{errorDelta}(\mathcal{D}, \mathbf{w}[j])$ for each weight is then the summation of the relevant column, for example, $\text{errorDelta}(\mathcal{D}, \mathbf{w}[0]) = 3,185.61$ and $\text{errorDelta}(\mathcal{D}, \mathbf{w}[1]) = 2,412,073.90$

Once the $\text{errorDelta}(\mathcal{D}, \mathbf{w}[j])$ for a weight has been calculated, we can then update the weight using Equation (7.17)^[340]. This weight update occurs on Line 4^[339] of Algorithm 7.1^[339]. The update involves multiplying the $\text{errorDelta}(\mathcal{D}, \mathbf{w}[j])$ for a given weight by the learning rate and then adding this to the current weight to give a new, updated, weight. The new set of weights is labeled **New Weights (after Iteration 1)** in Table 7.3^[345].

Table 7.3

Details of the first two iterations when the gradient descent algorithm is used to train a multivariable linear regression model for the office rentals dataset (using only the continuous descriptive features).

Initial Weights

$w[0]$:	-0.146	$w[1]$:	0.185	$w[2]$:	-0.044	$w[3]$:	0.119
----------	--------	----------	-------	----------	--------	----------	-------

Iteration 1

ID	RENTAL PRICE	Pred.	Error	Squared Error	$errorDelta(\mathcal{D}, w[j])$			
					$w[0]$	$w[1]$	$w[2]$	$w[3]$
1	320	93.26	226.74	51,411.08	226.74	113,370.05	906.96	1,813.92
2	380	107.41	272.59	74,307.70	272.59	149,926.92	1,908.16	13,629.72
3	400	115.15	284.85	81,138.96	284.85	176,606.39	2,563.64	1,993.94
4	390	119.21	270.79	73,327.67	270.79	170,598.22	1,353.95	6,498.98
5	385	134.64	250.36	62,682.22	250.36	166,492.17	2,002.91	25,036.42
6	410	130.31	279.69	78,226.32	279.69	195,782.78	1,118.76	2,237.52
7	480	142.89	337.11	113,639.88	337.11	259,570.96	3,371.05	2,359.74
8	600	168.32	431.68	186,348.45	431.68	379,879.24	5,180.17	21,584.05
9	570	170.63	399.37	159,499.37	399.37	367,423.83	5,591.23	3,194.99
10	620	187.58	432.42	186,989.95	432.42	432,423.35	3,891.81	10,378.16
Sum				1,067,571.59	3,185.61	2,412,073.90	27,888.65	88,727.43
Sum of squared errors (Sum/2)				533,785.80				

New Weights (after Iteration 1)

$w[0]$:	-0.146	$w[1]$:	0.233	$w[2]$:	-0.043	$w[3]$:	0.121
----------	--------	----------	-------	----------	--------	----------	-------

Iteration 2

ID	RENTAL PRICE	Pred.	Error	Squared Error	$errorDelta(\mathcal{D}, w[j])$			
					$w[0]$	$w[1]$	$w[2]$	$w[3]$
1	320	117.40	202.60	41,047.92	202.60	101,301.44	810.41	1,620.82
2	380	134.03	245.97	60,500.69	245.97	135,282.89	1,721.78	12,298.44
3	400	145.08	254.92	64,985.12	254.92	158,051.51	2,294.30	1,784.45
4	390	149.65	240.35	57,769.68	240.35	151,422.55	1,201.77	5,768.48
5	385	166.90	218.10	47,568.31	218.10	145,037.57	1,744.81	21,810.16
6	410	164.10	245.90	60,468.86	245.90	172,132.91	983.62	1,967.23
7	480	180.06	299.94	89,964.69	299.94	230,954.68	2,999.41	2,099.59
8	600	210.87	389.13	151,424.47	389.13	342,437.01	4,669.60	19,456.65
9	570	215.03	354.97	126,003.34	354.97	326,571.94	4,969.57	2,839.76
10	620	187.58	432.42	186,989.95	432.42	432,423.35	3,891.81	10,378.16
Sum				886,723.04	2,884.32	2,195,615.84	25,287.08	80,023.74
Sum of squared errors (Sum/2)				443,361.52				

New Weights (after Iteration 2)

$w[0]$:	-0.145	$w[1]$:	0.277	$w[2]$:	-0.043	$w[3]$:	0.123
----------	--------	----------	-------	----------	--------	----------	-------

We can see from Iteration 2 in the bottom half of Table 7.3^[345] that the new set of predictions made using the updated set of weights calculated in iteration 1 result in a lower sum of squared errors, 443,361.52. Based on this error another new set of weights is calculated using the error deltas shown. The algorithm then keeps iteratively applying the weight update rule until it converges on a stable set of weights beyond which little improvement in model accuracy is possible. In our example, convergence occurred after 100 iterations, and the final values for the weights were $w[0] = -0.1513$, $w[1] = 0.6270$, $w[2] = -0.1781$, and $w[3] = 0.0714$. The sum of squared errors for the final model was 2,913.5.¹⁰

A last point to make about this example is that careful examination of Table 7.3^[345] shows why such a low learning rate is used in this example. The large values of the RENTAL PRICE feature, [320, 620], causes the squared errors and, in turn, the error delta

values to become very large. This means that a very low **learning rate** is required in order to ensure that the changes made to the weights at each iteration of the learning process are small enough for the algorithm to work effectively. Using normalization (see Section 3.6.1^[92]) on the features can help avoid these large squared errors, and we do this in most examples from now on.

7.4 Extensions and Variations

In this section we discuss common and useful extensions to the basic multivariable linear regression with gradient descent approach described in Section 7.3^[332]. Topics covered include interpreting a linear regression model, using weight decay to set the learning rate, handling categorical descriptive and target features, using feature selection, using multivariable linear regression models to model non-linear relationships, and using support vector machines (SVMs) as an alternative to linear regression models.

Table 7.4

Weights and standard errors for each feature in the office rentals model.

Descriptive Feature	Weight	Standard Error	<i>t</i> -statistic	<i>p</i> -value
SIZE	0.6270	0.0545	11.504	< 0.0001
FLOOR	-0.178	12.7042	-0.066	0.949
BROADBAND RATE	0.071396	0.2969	0.240	0.816

7.4.1 Interpreting Multivariable Linear Regression Models

A particularly useful feature of linear regression models is that the weights used by the model indicate the effect of each descriptive feature on the predictions returned by the model. First, the signs of the weights indicate whether different descriptive features have a positive or a negative impact on the prediction. Table 7.4^[347] repeats the final weights for the office rentals model trained in Section 7.3.4^[343]. We can see that increasing office size leads to increasing rental prices; that lower building floors lead to higher rental prices; and that rental prices increase with broadband rates. Second, the magnitudes of the weights show how much the value of the target feature changes for a unit change in the value of a particular descriptive feature. For example, for every increase of a square foot in office size, we can expect the rental price to go up by 0.6270 Euro per month. Similarly, for every floor we go up in an office building, we can expect the rental price to decrease by 0.1781 Euro per month.

It is tempting to infer the relative importance of the different descriptive features in the model from the magnitude of the weights—i.e., the descriptive features associated with higher weights are more predictive than those with lower weights. This is a mistake, however, when the descriptive features themselves have varying scales. For example, in the office rentals dataset, the values of the SIZE feature range from 500 to 1,000 while the values for the FLOOR feature range from only 4 to 14. So, direct comparison of the weights tells us little about their relative importance. A better way to determine the importance of each descriptive feature in the model is to perform a **statistical significance test**.

A statistical significance test works by stating a **null hypothesis** and then determining whether there is enough evidence to accept or reject this hypothesis. This accept/reject decision is carried out in three steps:

1. A **test-statistic** is computed.
2. The probability of a test-statistic value as big as or greater than the one computed being the result of chance is calculated. This probability is called a **p-value**.
3. The **p-value** is compared to a predefined significance threshold, and if the **p-value** is less than or equal to the threshold (i.e., the **p-value** is small), the null hypothesis is rejected. These thresholds are typically the standard statistical thresholds of 5% or 1%.

The statistical significance test we use to analyze the importance of a descriptive feature $\mathbf{d}[j]$ in a linear regression model is the **t-test**. The null hypothesis that we adopt for this test is that the feature does not have a significant impact on the model. The test statistic we calculate is called the **t-statistic**. In order to calculate this test statistic, we first have to calculate the **standard error** for the overall model and the standard error for the descriptive feature we are investigating the importance of. The standard error for the overall model is calculated as

$$se = \sqrt{\frac{\sum_{i=1}^n (t_i - \mathbb{M}_w(\mathbf{d}_i))^2}{n-2}} \quad (7.18)$$

where n is the number of instances in the training dataset. A standard error calculation is then done for a descriptive feature as follows:

$$se(\mathbf{d}[j]) = \frac{se}{\sqrt{\sum_{i=1}^n (\mathbf{d}_i[j] - \overline{\mathbf{d}[j]})^2}} \quad (7.19)$$

where $\mathbf{d}[j]$ is some descriptive feature and $\overline{\mathbf{d}[j]}$ is the mean value of that descriptive feature in the training set.

The t -statistic for this test is calculated as follows:

$$t = \frac{\mathbf{w}[j]}{se(\mathbf{d}[j])} \quad (7.20)$$

where $\mathbf{w}[j]$ is the weight associated with descriptive feature $\mathbf{d}[j]$. Using a standard t -statistic look-up table, we can then determine the p -value associated with this test (this is a two tailed t -test with degrees of freedom set to the number of instances in the training set minus 2). If the p -value is less than the required significance level, typically 0.05, we reject the null hypothesis and say that the descriptive feature has a significant impact on the model; otherwise we say that it does not. We can see from Table 7.4^[347] that only the SIZE descriptive feature has a significant impact on the model. If a descriptive feature is found to have a significant impact on the model, this indicates that there is a significant linear relationship between it and the target feature.

7.4.2 Setting the Learning Rate Using Weight Decay

In Section 7.3.3^[341] we illustrated the impact of a learning rate parameter on the gradient descent algorithm. In that section we also explained that most practitioners use rules of thumb and trial and error to set the learning rate. A more systematic approach is to use learning rate decay, which allows the learning rate to start at a large value and then decay over time according to a predefined schedule. Although there are different approaches in the literature, a good approach is to use the following decay schedule:

$$\alpha_{\tau} = \alpha_0 \frac{c}{c + \tau} \quad (7.21)$$

where α_0 is an initial learning rate (this is typically quite large, e.g., 1.0), c is a constant that controls how quickly the learning rate decays (the value of this parameter depends on how quickly the algorithm converges, but it is often set to quite a large value, e.g., 100), and τ is the current iteration of the gradient descent algorithm. Figure 7.8^[350] shows the journey across the error surface and related plot of the sums of squared errors for the office rentals problem—using just the SIZE descriptive feature—when error decay is used with $\alpha_0 = 0.18$ and $c = 10$ (this is a pretty simple problem, so smaller values for these parameters are suitable). This example shows that the algorithm converges to the global minimum more quickly than any of the approaches shown in Figure 7.7^[342].

The differences between Figures 7.7(f)^[342] and 7.8(b)^[350] most clearly show the impact of learning rate decay as the initial learning rates are the same in these two instances. When learning rate decay is used, there is much less thrashing back and forth across the error surface than when the large static learning rate is used. Using learning rate decay can even address the problem of inappropriately large error rates causing the sum of squared errors to increase rather than decrease. Figure 7.9^[350] shows an example of this in which learning rate decay is used with $\alpha_0 = 0.25$ and $c = 100$. The algorithm starts at the position marked 1 on the error surface, and learning steps actually cause it to move farther and farther up the error surface. This can be seen in the increasing sums of squared errors in Figure 7.9(b)^[350]. As the learning rate decays, however, the direction of the journey across the error surface moves back downward, and eventually the global minimum is reached. Although learning rate decay almost always leads to better performance than a fixed learning rate, it still does require that problem-dependent values are chosen for α_0 and c .

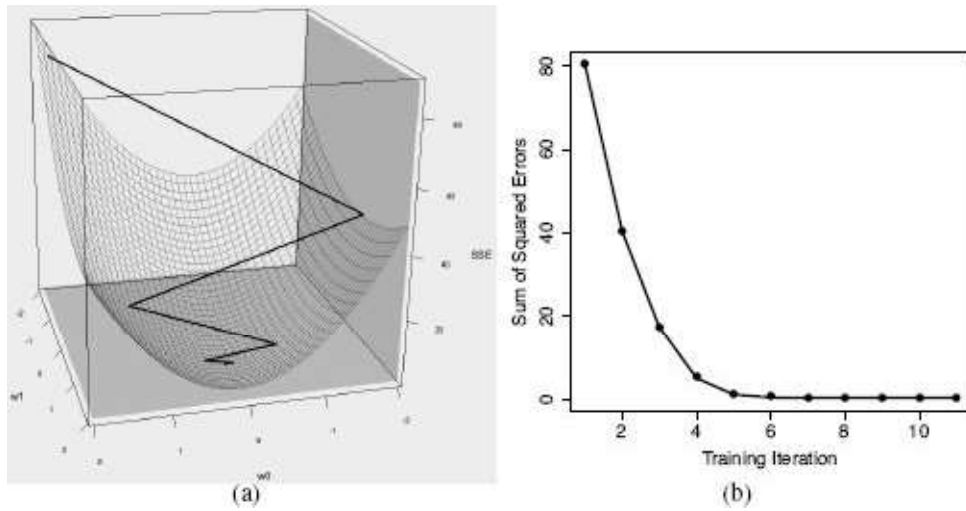


Figure 7.8

(a) The journey across the error surface for the office rentals prediction problem when learning rate decay is used ($\alpha_0 = 0.18$, $c = 10$); (b) a plot of the changing sum of squared errors during this journey.

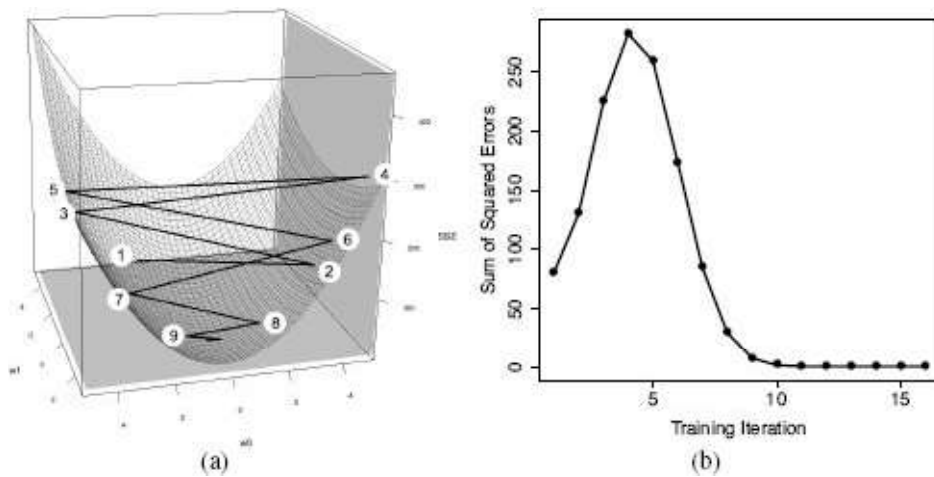


Figure 7.9

(a) The journey across the error surface for the office rentals prediction problem when learning rate decay is used ($\alpha_0 = 0.25$, $c = 100$); (b) a plot of the changing sum of squared errors during this journey.

7.4.3 Handling Categorical Descriptive Features

The regression equation for a multivariable linear regression model for the full dataset shown in Table 7.1^[325] would look like

$$\begin{aligned}\text{RENTAL PRICE} = & \mathbf{w}[0] + \mathbf{w}[1] \times \text{SIZE} + \mathbf{w}[2] \times \text{FLOOR} \\ & + \mathbf{w}[3] \times \text{BROADBAND RATE} \\ & + \mathbf{w}[4] \times \text{ENERGY RATING}\end{aligned}$$

The multiplication of $\mathbf{w}[4] \times \text{ENERGY RATING}$ causes a problem here. Energy rating is a categorical feature, so multiplying the values of this feature by a numeric weight is simply not sensible. The basic structure of the multivariable linear regression model allows for only continuous descriptive features. Obviously, though, in real-world datasets, we often encounter categorical descriptive features, so for the linear regression approach to be really useful, we need a way to handle these.

The most common approach to handling categorical features in linear regression models is to use a transformation that converts a single categorical descriptive feature into a number of continuous descriptive feature values that can encode the levels of the categorical feature. This is done by creating one new binary descriptive feature for every level of the categorical feature. These new features can then be used to encode a level of the original categorical descriptive feature by setting the value of the new feature corresponding to the level of the categorical feature to 1 and the other new continuous features to 0.

For example, if we were to use the ENERGY RATING descriptive feature from Table 7.1^[325] in a linear regression model, we would convert it into three new continuous descriptive features, as energy rating can have one of three distinct levels: A, B, or C. Table 7.5^[352] shows this transformed dataset in which the energy rating feature has been replaced with ENERGY RATING A, ENERGY RATING B and ENERGY RATING C. For training instances in which the original ENERGY RATING feature had a value A, the new ENERGY RATING A feature has a value of 1, and the ENERGY RATING B and ENERGY RATING C are both set to 0. A similar rule is used for instances with the ENERGY RATING feature levels of B and C.

Table 7.5

The office rentals dataset from Table 7.1^[325] adjusted to handle the categorical ENERGY RATING descriptive feature in linear regression models.

ID	SIZE	FLOOR	BROADBAND RATE	ENERGY RATING A	ENERGY RATING B	ENERGY RATING C	RENTAL PRICE
1	500	4	8	0	0	1	320
2	550	7	50	1	0	0	380
3	620	9	7	1	0	0	400
4	630	5	24	0	1	0	390
5	665	8	100	0	0	1	385
6	700	4	8	0	1	0	410
7	770	10	7	0	1	0	480
8	880	12	50	1	0	0	600
9	920	14	8	0	0	1	570
10	1,000	9	24	0	1	0	620

Returning to our example, the regression equation for this RENTAL PRICE model would change to

$$\begin{aligned}
 \text{RENTAL PRICE} = & w[0] + w[1] \times \text{SIZE} + w[2] \times \text{FLOOR} \\
 & + w[3] \times \text{BROADBAND RATE} \\
 & + w[4] \times \text{ENERGY RATING A} \\
 & + w[5] \times \text{ENERGY RATING B} \\
 & + w[6] \times \text{ENERGY RATING C}
 \end{aligned}$$

where the newly added categorical features allow the original ENERGY RATING feature to be included. Everything else about using such a model is exactly the same as before.

The downside to this approach is that it introduces a number of extra weights for which optimal values must be found—in this simple example for only four descriptive features, we need seven weights. This increases the size of the weight space through which we need to search when training the model. One way we can reduce the impact of this is that for each categorical feature we transform, we can reduce the number of newly added features by one by assuming that a zero in all the new features implies that the original feature had the final level. So, for example, for our ENERGY RATING feature, instead of adding three new features (ENERGY RATING A, ENERGY RATING B, and ENERGY RATING C), we could just add ENERGY RATING A and ENERGY RATING B and assume that whenever they both have a value of 0, ENERGY RATING C is implicitly set.

7.4.4 Handling Categorical Target Features: Logistic Regression

In Section 7.3^[332] we described how a multivariable linear regression model trained using gradient descent can be used to make predictions for continuous target features. Although this is useful for a range of real-world predictive analytics problems, we are also interested in prediction problems with categorical target features. This section covers the reasonably simple adjustments that must be made to the multivariable linear regression with gradient descent algorithm to handle categorical target features, in particular, logistic regression.

7.4.4.1 Predicting Categorical Targets Using Linear Regression

Table 7.6^[354] shows a sample dataset with a categorical target feature. This dataset contains measurements of the revolutions per minute (RPM) that power station generators are running at, the amount of vibration in the generators (VIBRATION), and an indicator to show whether the generators proved to be working or faulty the day after these measurements were taken. The RPM and VIBRATION measurements come from the day before the generators proved to be operational or faulty. If power station administrators could predict upcoming generator failures before the generators actually fail, they could improve power station safety and save money on maintenance.¹¹ Using this dataset, we would like to train a model to distinguish between properly operating power station generators and faulty generators using the RPM and VIBRATION measurements.

Figure 7.10(a)^[355] shows a scatter plot of this dataset in which we can see that there is a good separation between the two types of generator. In fact, as shown in Figure 7.10(b)^[355], we can draw a straight line across the scatter plot that perfectly separates the *good* generators from the *faulty* ones. This line is known as a decision boundary, and because we can draw this line, this dataset is said to be linearly separable in terms of the two descriptive features used. As the decision boundary is a linear separator, it can be defined using the equation of the line (remember Equation (7.2.1)^[325]). In Figure 7.10(b)^[355] the decision boundary is defined as

$$\text{VIBRATION} = 830 - 0.667 \times \text{RPM} \quad (7.22)$$

or

$$830 - 0.667 \times \text{RPM} - 1 \times \text{VIBRATION} = 0 \quad (7.23)$$

Table 7.6

A dataset listing features for a number of generators.

ID	RPM	VIBRATION	STATUS
1	568	585	good
2	586	565	good
3	609	536	good

4	616	492	good
5	632	465	good
6	652	528	good
7	655	496	good
8	660	471	good
9	688	408	good
10	696	399	good
11	708	387	good
12	701	434	good
13	715	506	good
14	732	485	good
15	731	395	good
16	749	398	good
17	759	512	good
18	773	431	good
19	782	456	good
20	797	476	good
21	794	421	good
22	824	452	good
23	835	441	good
24	862	372	good
25	879	340	good
26	892	370	good

27	913	373	good
28	933	330	good
29	562	309	faulty
30	578	346	faulty
31	593	357	faulty
32	626	341	faulty
33	635	252	faulty
34	658	235	faulty
35	663	299	faulty
36	677	223	faulty
37	685	303	faulty
38	698	197	faulty
39	699	311	faulty
40	712	257	faulty
41	722	193	faulty
42	735	259	faulty
43	738	314	faulty
44	753	113	faulty
45	767	286	faulty
46	771	264	faulty
47	780	137	faulty
48	784	131	faulty
49	798	132	faulty

50	820	152	faulty
51	834	157	faulty
52	858	163	faulty
53	888	91	faulty
54	891	156	faulty
55	911	79	faulty
56	939	99	faulty

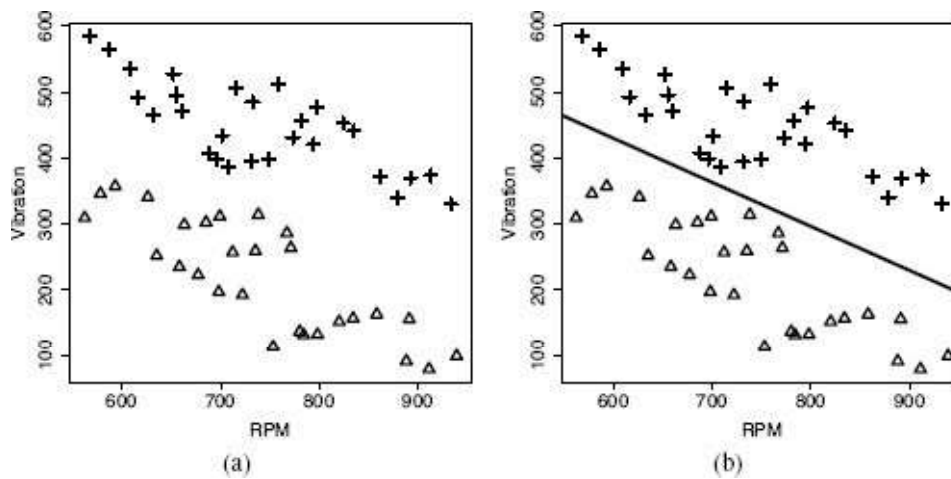


Figure 7.10

(a) A scatter plot of the RPM and VIBRATION descriptive features from the generators dataset shown in Table 7.6^[354], where *good* generators are shown as crosses, and *faulty* generators are shown as triangles; (b) as decision boundary separating *good* generators (crosses) from *faulty* generators (triangles).

So, for any instance that is actually on the decision boundary, the RPM and VIBRATION values satisfy the equality in Equation (7.23)^[354]. What is more interesting is that instances not actually on the decision boundary behave in a very regular way. The descriptive feature values of all instances above the decision boundary will result in a negative value when plugged into the decision boundary equation, while the descriptive features of all instances below the decision boundary will result in a positive value. For example, applying Equation (7.23)^[354] to the instance RPM = 810, VIBRATION = 495, which is above the decision boundary in Figure 7.10(b)^[355], gives the following result:

$$830 - 0.667 \times 810 - 495 = -205.27$$

By contrast, if we apply Equation (7.23)^[354] to the instance RPM = 650 and VIBRATION =

240, which is below the decision boundary in Figure 7.10(b)^[355], we get

$$830 - 0.667 \times 650 - 240 = 156.45$$

Figure 7.11(a)^[356] illustrates the consistent relationship between Equation (7.23)^[354] and the decision boundary by plotting the value of Equation (7.23)^[354] for all values of RPM and VIBRATION.¹²

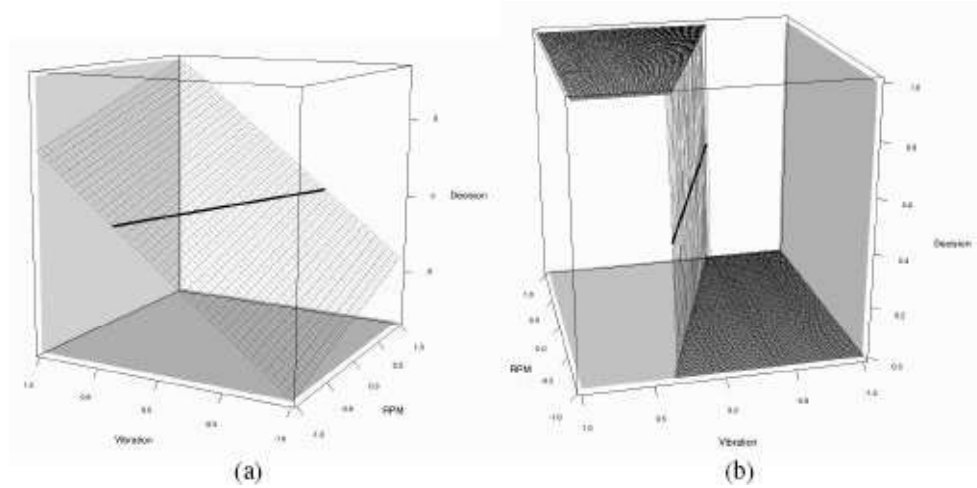


Figure 7.11

(a) A surface showing the value of Equation (7.23)^[354] for all values of RPM and VIBRATION, with the decision boundary given in Equation (7.23)^[354] highlighted; (b) the same surface linearly thresholded at zero to operate as a predictor.

Because the values of this equation are so well behaved, we can use it to predict a categorical target feature. Reverting to our previous notation, we have,

$$\mathbb{M}_{\mathbf{w}}(\mathbf{d}) = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{d} \geq 0 \\ 0 & \text{otherwise} \end{cases} \quad (7.24)$$

where $\mathbf{d}$ is a set of descriptive features for an instance, $\mathbf{w}$ is the set of weights in the model, and the *good* and *faulty* generator target feature levels are represented as 0 and 1 respectively. Figure 7.11(b)^[356] shows the value of Equation (7.24)^[356] for every possible value of RPM and VIBRATION. This surface is known as a **decision surface**.

One problem that we need to solve in order to use the model defined in Equation (7.24)^[356] is how to determine the values for the weights, $\mathbf{w}$, that will minimize the error function for our hypothesis $\mathbb{M}_{\mathbf{w}}(\mathbf{d})$. Unfortunately, in this case we cannot just use the gradient descent algorithm. The hard decision boundary given in Equation (7.24)^[356] is **discontinuous**, so is not differentiable, which means we cannot calculate the gradient of the error surface using the derivative. Another problem with this model is that the model always makes completely confident predictions of 0 or 1. A model able to distinguish between instances that are very close to the boundary and those that are farther away would be preferable. We can solve both these problems by using a more sophisticated threshold function that is continuous, and therefore differentiable, and that allows for the

subtlety desired: the **logistic function**.¹³

The logistic function¹⁴ is given by

$$\text{logistic}(x) = \frac{1}{1 + e^{-x}} \quad (7.25)$$

where x is a numeric value and e is Euler's number and is approximately equal to 2.7183. A plot of the logistic function for values of x in the range $[-10, 10]$ is shown in Figure 7.12(a)^[358]. We can see that the logistic function is a threshold function that pushes values above zero to 1 and values below zero to 0. This is very similar to the hard threshold function given in Equation (7.24)^[356], except that it has a soft boundary. The next section explains how use of the logistic function allows us to build logistic regression models that predict categorical target features.

7.4.4.2 Logistic Regression

To build a logistic regression model, we threshold the output of the basic linear regression model using the logistic function. So, instead of the regression function simply being the dot product of the weights and the descriptive features (as given in Equation (7.9)^[333]), the dot product of weights and descriptive feature values is passed through the logistic function:

$$\begin{aligned} \mathbb{M}_{\mathbf{w}}(\mathbf{d}) &= \text{logistic}(\mathbf{w} \cdot \mathbf{d}) \\ &= \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{d}}} \end{aligned} \quad (7.26)$$

To see the impact of this, we can build a multivariable logistic regression model for the dataset in Table 7.6^[354]. After the training process (which uses a slightly modified version of the gradient descent algorithm, which we will explain shortly), the resulting logistic regression model is¹⁵

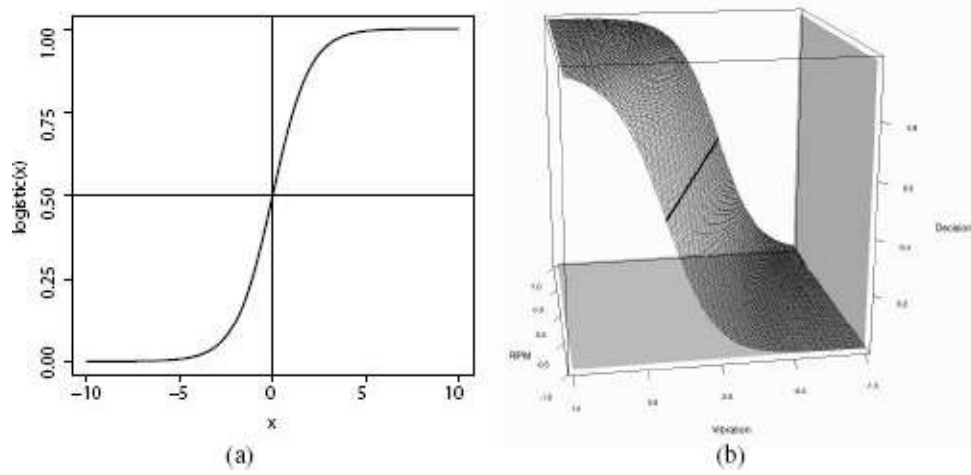


Figure 7.12

(a) A plot of the logistic function (Equation (7.25)^[357]) for the range of values $[-10, 10]$; (b) the logistic decision surface that results from training a model to represent the generators dataset given in Table 7.6^[354] (note that the data has been normalized to the range $[-1, 1]$).

$$\mathbb{M}_w(\langle \text{RPM}, \text{VIBRATION} \rangle) = \frac{1}{1 + e^{-(-0.4077 + 4.1697 \times \text{RPM} + 6.0460 \times \text{VIBRATION})}} \quad (7.27)$$

The decision surface resulting from Equation (7.27)^[358] is shown in Figure 7.12(b)^[358]. The important thing to notice about this decision surface, in contrast to the decision surface in Figure 7.11(b)^[356], is that there is a gentle transition from predictions of the *faulty* target level to predictions of the *good* generator target level. This is one of the key benefits of using logistic regression. Another benefit of using the logistic function is that logistic regression model outputs can be interpreted as probabilities of the occurrence of a target level. So

$$P(t = \text{faulty} | \mathbf{d}) = \mathbb{M}_w(\mathbf{d})$$

and

$$P(t = \text{good} | \mathbf{d}) = 1 - \mathbb{M}_w(\mathbf{d})$$

To find the optimal decision boundary for a logistic regression problem, we use the gradient descent algorithm (Algorithm 7.1^[339]) to minimize the sum of squared errors based on the training dataset. Figure 7.13^[359] shows a series of the candidate models that were explored on the way to finding this boundary. The final panel in Figure 7.13^[359] shows how the sum of squared errors changed during the training process.

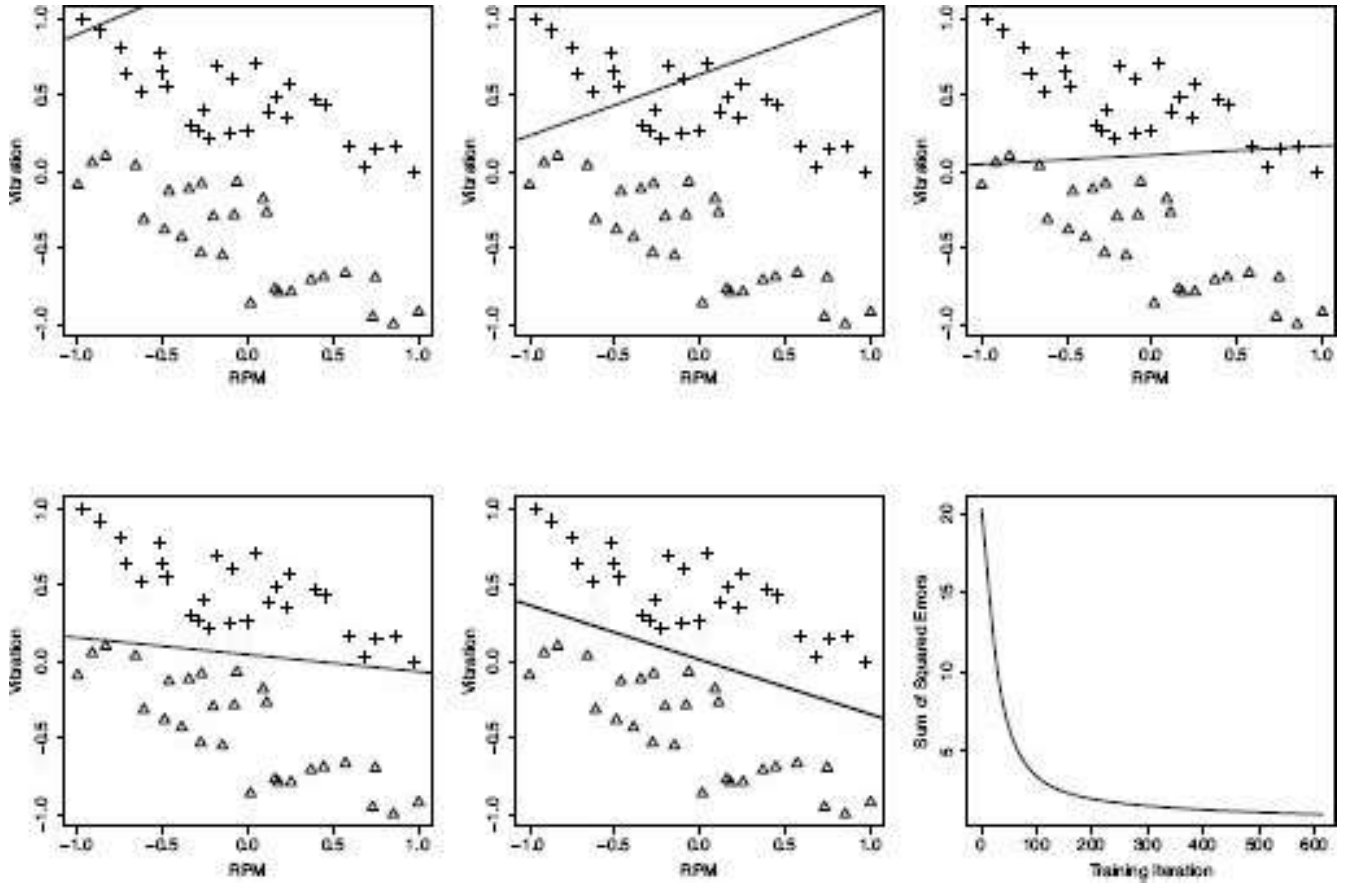


Figure 7.13

A selection of the logistic regression models developed during the gradient descent process for the machinery dataset from Table 7.6^[354]. The bottom-right panel shows the sums of squared errors generated during the gradient descent process.

To repurpose the gradient descent algorithm for training logistic regression models, the only change that needs to be made is in the error delta function, which is used in the weight update rule given on Line 4^[339] of Algorithm 7.1^[339]. To derive this new weight update rule, imagine that there is just a single training instance, $(\mathbf{d}, t)$, in our training dataset. The partial derivative of the error function, L_2 , is then

$$\frac{\partial}{\partial \mathbf{w}[j]} L_2(\mathbb{M}_{\mathbf{w}}, \mathcal{D}) = \frac{\partial}{\partial \mathbf{w}[j]} \frac{1}{2} (t - \mathbb{M}_{\mathbf{w}}(\mathbf{d}))^2$$

where $\mathbf{w}[j]$ is a single weight from the set of weights $\mathbf{w}$. Applying the chain rule to this, we get

$$\frac{\partial}{\partial \mathbf{w}[j]} L_2(\mathbb{M}_{\mathbf{w}}, \mathcal{D}) = (t - \mathbb{M}_{\mathbf{w}}(\mathbf{d})) \times \frac{\partial}{\partial \mathbf{w}[j]} (t - \mathbb{M}_{\mathbf{w}}(\mathbf{d}))$$

But $\mathbb{M}_{\mathbf{w}}(\mathbf{d}) = \text{logistic}(\mathbf{w} \cdot \mathbf{d})$, so

$$\frac{\partial}{\partial \mathbf{w}[j]} L_2(\mathbb{M}_{\mathbf{w}}, \mathcal{D}) = (t - \text{logistic}(\mathbf{w} \cdot \mathbf{d})) \times \frac{\partial}{\partial \mathbf{w}[j]} (t - \text{logistic}(\mathbf{w} \cdot \mathbf{d}))$$

Applying the chain rule again to the partial derivative part of this equation, and remembering that $\frac{\partial}{\partial \mathbf{w}[j]} \mathbf{w} \cdot \mathbf{d} = d[j]$, we get

$$\begin{aligned}
\frac{\partial}{\partial \mathbf{w}[j]} L_2(\mathbb{M}_{\mathbf{w}}, \mathcal{D}) &= (t - \text{logistic}(\mathbf{w} \cdot \mathbf{d})) \times \frac{\partial}{\partial \mathbf{w}[j]} \text{logistic}(\mathbf{w} \cdot \mathbf{d}) \\
&\quad \times \frac{\partial}{\partial \mathbf{w}[j]} \mathbf{w} \cdot \mathbf{d} \\
&= (t - \mathbb{M}_{\mathbf{w}}(\mathbf{d})) \times \frac{\partial}{\partial \mathbf{w}[j]} \text{logistic}(\mathbf{w} \cdot \mathbf{d}) \times \mathbf{d}[j]
\end{aligned}$$

Fortunately, the derivative of the logistic function is well known:

$$\frac{d}{dx} \text{logistic}(x) = \text{logistic}(x)(1 - \text{logistic}(x)) \quad (7.28)$$

So

$$\begin{aligned}
\frac{\partial}{\partial \mathbf{w}[j]} L_2(\mathbb{M}_{\mathbf{w}}, \mathcal{D}) &= (t - \text{logistic}(\mathbf{w} \cdot \mathbf{d})) \\
&\quad \times \text{logistic}(\mathbf{w} \cdot \mathbf{d})(1 - \text{logistic}(\mathbf{w} \cdot \mathbf{d})) \times \mathbf{d}[j]
\end{aligned} \quad (7.29)$$

Rewriting $\text{logistic}(\mathbf{w} \cdot \mathbf{d})$ as $\mathbb{M}_{\mathbf{w}}(\mathbf{d})$ for readability, we get

$$\frac{\partial}{\partial \mathbf{w}[j]} L_2(\mathbb{M}_{\mathbf{w}}, \mathcal{D}) = (t - \mathbb{M}_{\mathbf{w}}(\mathbf{d})) \quad (7.30)$$

$$\times \mathbb{M}_{\mathbf{w}}(\mathbf{d}) \times (1 - \mathbb{M}_{\mathbf{w}}(\mathbf{d})) \times \mathbf{d}[j] \quad (7.31)$$

This is the partial derivative of the error surface with respect to a particular weight $\mathbf{w}[j]$ and indicates the gradient of the error surface. Using this formulation for the gradient, we can write the weight update rule for logistic regression as

$$\mathbf{w}[j] \leftarrow \mathbf{w}[j] + \alpha \times (t - \mathbb{M}_{\mathbf{w}}(\mathbf{d})) \times \mathbb{M}_{\mathbf{w}}(\mathbf{d}) \times (1 - \mathbb{M}_{\mathbf{w}}(\mathbf{d})) \times \mathbf{d}[j] \quad (7.32)$$

where $\mathbb{M}_{\mathbf{w}}(\mathbf{d}) = \text{logistic}(\mathbf{w} \cdot \mathbf{d}) = \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{d}}}$.

The rule given in Equation (7.32)^[361] assumes that only a single training instance exists. To modify this to take into account a full training dataset, we simply need to a sum across all the training instances as we did before in Equation (7.17)^[340]. This gives us the weight update rule for multivariable logistic regression:

$$\mathbf{w}[j] \leftarrow \mathbf{w}[j] + \alpha \times \sum_{i=1}^n ((t_i - \mathbb{M}_{\mathbf{w}}(\mathbf{d}_i)) \times \mathbb{M}_{\mathbf{w}}(\mathbf{d}_i) \times (1 - \mathbb{M}_{\mathbf{w}}(\mathbf{d}_i)) \times \mathbf{d}_i[j]) \quad (7.33)$$

Other than changing the weight update rule, we don't need to make any other changes to the model training process presented for multivariable linear regression models. To further illustrate this process, the next section presents a worked example of training a multivariable logistic regression model for an extended version of the generators dataset.

7.4.4.3 A Worked Example of Multivariable Logistic Regression

One of the advantages of using a logistic regression model is that it works well for datasets in which the instances with target features set to different levels overlap with each other in the feature space. Table 7.7^[362] shows an extended version of the generators dataset given in Table 7.6^[354], including extra instances that make the separation between

good generators and *faulty* generators less clear cut. This kind of data is very common in real-world scenarios. A scatter plot of this dataset is shown in Figure 7.14^[363],s in which the overlap between the different types of generator in this dataset is clearly visible. Even though the separation between the instances with the different levels of the target feature in this case is not particularly well defined, a logistic regression model can be trained to distinguish between the two types of generator. In the remainder of this section, we examine this in some detail.

There is an on-going argument regarding whether descriptive features should be **normalized** before being used in linear regression models. The main disadvantage of normalization is that the interpretative analysis discussed in Section 7.4.4^[353] becomes more difficult as the descriptive feature values used in the model do not relate to the actual feature values in the data. For example, if the age of a customer was used as a descriptive feature in a financial credit scoring model, it is more difficult to talk about changes in normalized age on a scale from 0 to 1 than it is to discuss original age values on their natural scale, about 18 to 80. The main advantages of normalizing descriptive feature values are that all weights become directly comparable with each other (as all descriptive features are on the same scale), and the behavior of the gradient descent algorithm used to train the model becomes much less sensitive to the learning rate and the initial weights. Although it is less important for simple linear regression models, for logistic regression models we recommend that descriptive feature values always be normalized. In this example, before the training process begins, both descriptive features are normalized to the range $[-1, 1]$.

Table 7.7

An extended version of the generators dataset from Table 7.6^[354].

ID	RPM	VIBRATION	STATUS
1	498	604	faulty
2	517	594	faulty
3	541	574	faulty
4	555	587	faulty
5	572	537	faulty
6	600	553	faulty
7	621	482	faulty
8	632	539	faulty

9	656	476	faulty
10	653	554	faulty
11	679	516	faulty
12	688	524	faulty
13	684	450	faulty
14	699	512	faulty
15	703	505	faulty
16	717	377	faulty
17	740	377	faulty
18	749	501	faulty
19	756	492	faulty
20	752	381	faulty
21	762	508	faulty
22	781	474	faulty
23	781	480	faulty
24	804	460	faulty
25	828	346	faulty
26	830	366	faulty
27	864	344	faulty
28	882	403	faulty
29	891	338	faulty
30	921	362	faulty
31	941	301	faulty

32	965	336	faulty
33	976	297	faulty
34	994	287	faulty
35	501	463	good
36	526	443	good
37	536	412	good
38	564	394	good
39	584	398	good
40	602	398	good
41	610	428	good
42	638	389	good
43	652	394	good
44	659	336	good
45	662	364	good
46	672	308	good
47	691	248	good
48	694	401	good
49	718	313	good
50	720	410	good
51	723	389	good
52	744	227	good
53	741	397	good
54	770	200	good

55	764	370	good
56	790	248	good
57	786	344	good
58	792	290	good
59	818	268	good
60	845	232	good
61	867	195	good
62	878	168	good
63	895	218	good
64	916	221	good
65	950	156	good
66	956	174	good
67	973	134	good
68	1002	121	good

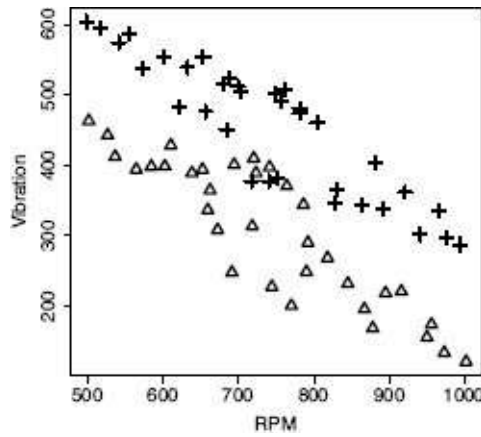


Figure 7.14

A scatter plot of the extended generators dataset given in Table 7.7^[362], which results in instances with the different target levels overlapping with each other. Instances representing *good* generators are shown as crosses, and those representing *faulty* generators as triangles.

To begin the gradient descent process, random starting values for the weights within the model, $w[0]$, $w[1]$, $w[2]$, are selected. In this example, random values were selected from the range $[-3, 3]$ to give $w[0] = -2.9465$, $w[1] = -1.0147$, and $w[2] = 2.1610$. Using these weights, a prediction is made for every instance in the training dataset, and the resulting sum of squared errors is calculated. The predictions made using these weights and the related error are shown in Table 7.8^[364] under **Iteration 1**.

Table 7.8

Details of the first two iterations when the gradient descent algorithm is used to train a logistic regression model for the extended generators dataset given in Table 7.7^[362].

Initial Weights

$w[0]:$	-2.9465	$w[1]:$	-1.0147	$w[2]:$	-2.1610
---------	---------	---------	---------	---------	---------

Iteration 1

ID	TARGET LEVEL	Pred.	Error	Squared Error	$errorDelta(\mathcal{D}, w[j])$		
					$w[0]$	$w[1]$	$w[2]$
1	1	0.5570	0.4430	0.1963	0.1093	-0.1093	0.1093
2	1	0.5168	0.4832	0.2335	0.1207	-0.1116	0.1159
3	1	0.4469	0.5531	0.3059	0.1367	-0.1134	0.1197
...							
66	0	0.0042	-0.0042	0.0000	0.0000	0.0000	0.0000
67	0	0.0028	-0.0028	0.0000	0.0000	0.0000	0.0000
68	0	0.0022	-0.0022	0.0000	0.0000	0.0000	0.0000
Sum				24.4738	2.7031	-0.7015	1.6493
Sum of squared errors (Sum/2)				12.2369			

New Weights (after Iteration 1)

$w[0]:$	-2.8924	$w[1]:$	-1.0287	$w[2]:$	-2.1940
---------	---------	---------	---------	---------	---------

Iteration 2

ID	TARGET LEVEL	Pred.	Error	Squared Error	$errorDelta(\mathcal{D}, w[j])$		
					$w[0]$	$w[1]$	$w[2]$
1	1	0.5817	0.4183	0.1749	0.1018	-0.1018	0.1018
2	1	0.5414	0.4586	0.2103	0.1139	-0.1053	0.1094
3	1	0.4704	0.5296	0.2805	0.1319	-0.1094	0.1155
...							
66	0	0.0043	-0.0043	0.0000	0.0000	0.0000	0.0000
67	0	0.0028	-0.0028	0.0000	0.0000	0.0000	0.0000
68	0	0.0022	-0.0022	0.0000	0.0000	0.0000	0.0000
Sum				24.0524	2.7236	-0.6646	1.6484
Sum of squared errors (Sum/2)				12.0262			

New Weights (after Iteration 2)

$w[0]:$	-2.8380	$w[1]:$	-1.0416	$w[2]:$	-2.2271
---------	---------	---------	---------	---------	---------

This first candidate model is not particularly accurate with an initial sum of squared errors of 12.2369. In fact, instances 1 and 2 are the only instances at this stage that are given predictions of the *faulty* target level, level 1 (note that their prediction values are the only ones greater than 0.5). This can also be seen in the top left hand image of Figure 7.15^[366], which shows the candidate model corresponding to this initial set of weights. Based on the errors in these predictions, the delta contributions, labeled as $errorDelta(\mathcal{D}, w[0])$, $errorDelta(\mathcal{D}, w[1])$ and $errorDelta(\mathcal{D}, w[2])$ in Table 7.8^[364], from each training instance are calculated according to Equation (7.31)^[360]. These individual delta

contributions are then summed so that the weight update rule (Equation (7.33)^[361]) can be applied, in this example using a learning rate of 0.02. So, for example, the new value of $w[0]$ is calculated as the old value plus the learning rate times the sum of the *errorDelta*($\mathcal{D}$, $w[0]$) contributions to give $-2.9465 + 0.02 \times 2.7031 = -2.8924$. This gives the new set of weights shown as **New Weights (after Iteration 1)**.

The process then starts again using these new weights as the basis for the predictions and errors marked as **Iteration 2** in Table 7.8^[364]. The new weights result in slightly more accurate predictions, evident from the slightly reduced sum of squared errors of 12.0262. Based on the updated errors, a new set of weights is calculated, marked in Table 7.8^[364] as **New Weights (after Iteration 2)**. Table 7.8^[364] shows just the first two iterations of the gradient descent process for this model. The continuing process that finds the final model is illustrated in Figure 7.15^[366], which shows a selection of the candidate models generated on the way to generating the final model, and the bottom-right panel shows how the sum of squared errors changed during the process. The final model trained is

$$M_w(\langle \text{RPM}, \text{VIBRATION} \rangle) = \frac{1}{1 + e^{-(-0.4077 + 4.1697 \times \text{RPM} + 6.0460 \times \text{VIBRATION})}}$$

which has a sum of squared errors of 1.8804. Obviously, because there are instances with different levels for the target feature overlapping in the feature space, it is not possible in this case to build a model that perfectly separates the *good* and *faulty* machines. The model trained, however, strikes a good balance between mistaking *good* machines for *faulty* ones and vice versa.

7.4.5 Modeling Non-linear Relationships

All the simple linear regression and logistic regression models that we have looked at so far model a linear relationship between descriptive features and a target feature. Linear models work very well when the underlying relationships in the data are linear. Sometimes, however, the underlying data will exhibit non-linear relationships that we would like to capture in a model. For example, the dataset in Table 7.9^[367] is based on an agricultural scenario and shows rainfall (in mm per day), RAIN, and resulting grass growth (in kilograms per acre per day), GROWTH, measured on a number of Irish farms during July 2012. A scatter plot of these two features is shown in Figure 7.16(a)^[368], from which the strong non-linear relationship between rainfall and grass growth is clearly apparent—grass does not grow well when there is very little rain or too much rain, but hits a sweet spot at rainfall of about 2.5mm per day. It would be useful for farmers to be able to predict grass growth for different amounts of forecasted rainfall so that they could plan the optimal times to harvest their grass for making hay.

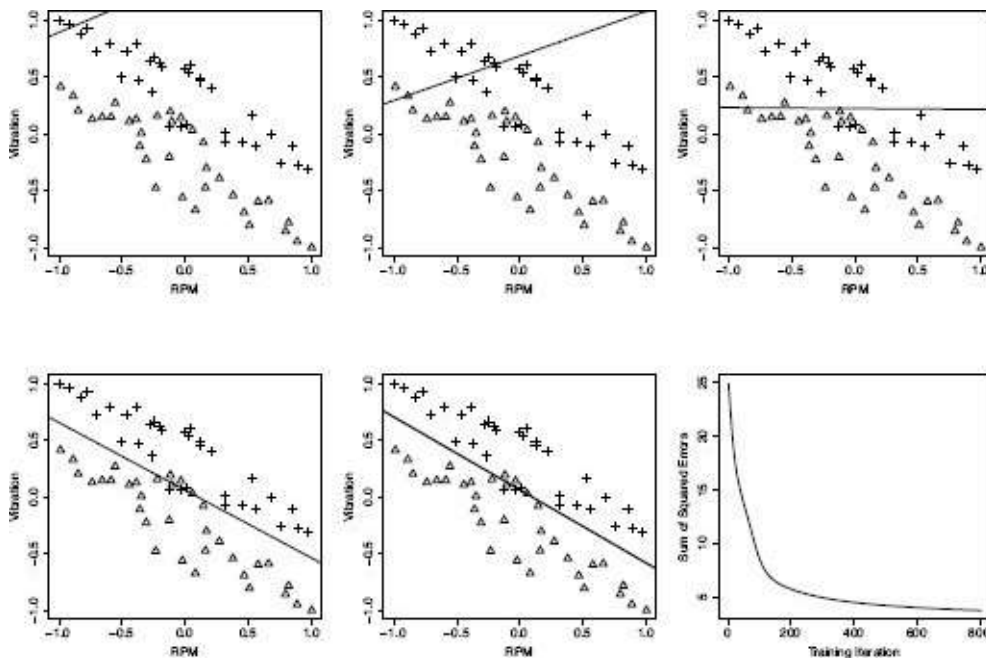


Figure 7.15

A selection of the logistic regression models developed during the gradient descent process for the extended generators dataset in Table 7.7^[362]. The bottom-right panel shows the sums of squared errors generated during the gradient descent process.

A simple linear regression model cannot handle this non-linear relationship. Figure 7.16(b)^[368] shows the best simple linear regression model that can be trained for this prediction problem. This model is

$$\text{GROWTH} = 13.510 + -0.667 \times \text{RAIN}$$

To successfully model the relationship between grass growth and rainfall, we need to introduce non-linear elements. A generalized way in which to do this is to introduce basis functions that transform the raw inputs to the model into non-linear representations but

still keep the model itself linear in terms of the weights. The advantage of this is that, except for introducing the mechanism of basis functions, we do not need to make any other changes to the approach we have presented so far. Furthermore, basis functions work for both simple multivariable linear regression models that predict a continuous target feature and multivariable logistic regression models that predict a categorical target feature.

Table 7.9

A dataset describing grass growth on Irish farms during July 2012.

ID	RAIN	GROWTH
1	2.153	14.016
2	3.933	10.834
3	1.699	13.026
4	1.164	11.019
5	4.793	4.162
6	2.690	14.167
7	3.982	10.190
8	3.333	13.525
9	1.942	13.899
10	2.876	13.949
11	4.277	8.643
12	3.754	11.420
13	2.809	13.847
14	1.809	13.757
15	4.114	9.101
16	2.834	13.923
17	3.872	10.795

18	2.174	14.307
19	4.353	8.059
20	3.684	12.041
21	2.140	14.641
22	2.783	14.138
23	3.960	10.307
24	3.592	12.069
25	3.451	12.335
26	1.197	10.806
27	0.723	7.822
28	1.958	14.010
29	2.366	14.088
30	1.530	12.701
31	0.847	9.012
32	3.843	10.885
33	0.976	9.876

To use basis functions, we recast the simple linear regression model (see Equation (7.9) [333]) as follows:

$$\mathbb{M}_{\mathbf{w}}(\mathbf{d}) = \sum_{k=0}^b \mathbf{w}[k] \times \phi_k(\mathbf{d}) \quad (7.34)$$

where $\mathbf{d}$ is a set of m descriptive features, $\mathbf{w}$ is a set of b weights, and ϕ_0 to ϕ_b are a series of b basis functions that each transform the input vector $\mathbf{d}$ in a different way. It is worth noting that there is no reason that b must equal m , and usually b is quite a bit larger than m —i.e., there are usually more basis functions than there are descriptive features.

One of the most common uses of basis functions in linear regression is to train models

to capture **polynomial** relationships. A linear relationship implies that the target is calculated from the descriptive features using only the addition of the descriptive feature values multiplied by weight values. Polynomial relationships allow multiplication of descriptive feature values by each other and raising of descriptive features to exponents. The most common form of polynomial relationship is the **second order polynomial**, also known as the **quadratic function**, which takes the general form $a = bx + cx^2$. The relationship between rainfall and grass growth in the grass growth dataset can be accurately represented as a second order polynomial through the following model:

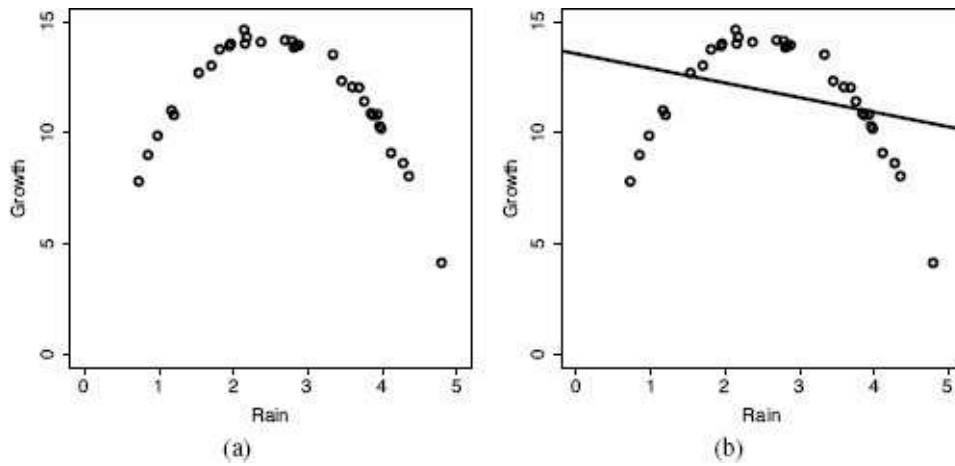


Figure 7.16

(a) A scatter plot of the RAIN and GROWTH feature from the grass growth dataset; (b) the same plot with a simple linear regression model trained to capture the relationship between the grass growth and rainfall.

$$\text{GROWTH} = \mathbf{w}[0] \times \phi_0(\text{RAIN}) + \mathbf{w}[1] \times \phi_1(\text{RAIN}) + \mathbf{w}[2] \times \phi_2(\text{RAIN})$$

where

$$\begin{aligned}\phi_0(\text{RAIN}) &= 1 \\ \phi_1(\text{RAIN}) &= \text{RAIN} \\ \phi_2(\text{RAIN}) &= \text{RAIN}^2\end{aligned}$$

What makes this approach really attractive is that, although this new model stated in terms of basis functions captures the non-linear relationship between rainfall and grass growth, the model is still linear in terms of the weights and so can be trained using gradient descent without making any changes to the algorithm. Figure 7.17^[369] shows the final non-linear model that results from this training process, along with a number of the interim steps on the way to this model. The final model is

$$\text{GROWTH} = 3.707 \times \phi_0(\text{RAIN}) + 8.475 \times \phi_1(\text{RAIN}) + -1.717 \times \phi_2(\text{RAIN})$$

where ϕ_0 , ϕ_1 , and ϕ_2 are as described before. This model captures the non-linear relationship in the data very well but was still easy to train using a gradient descent approach. Basis functions can also be used for multivariable simple linear regression models in the same way, the only extra requirement being the definition of more basis

functions.

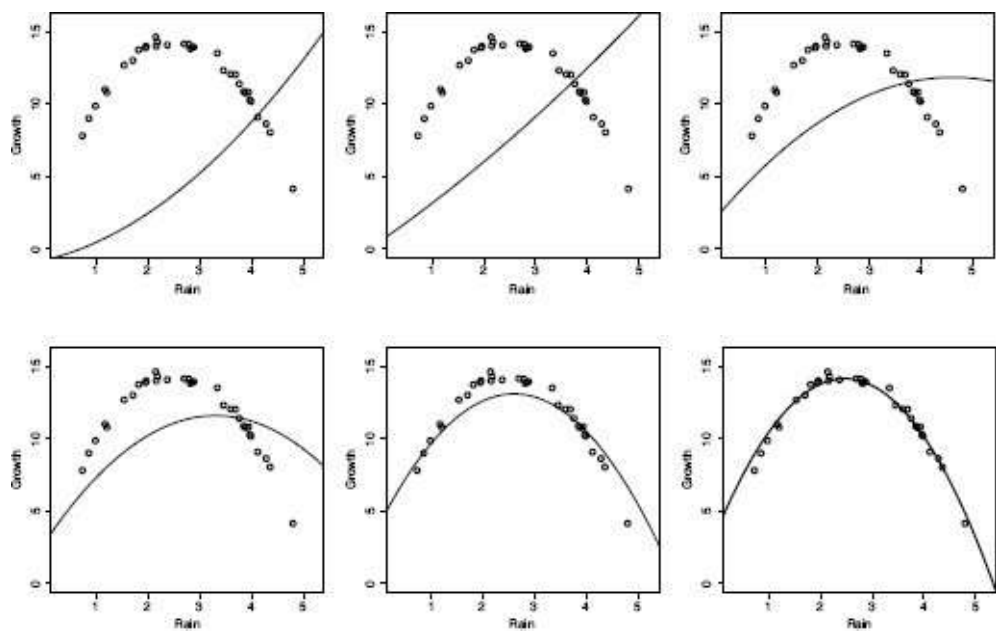


Figure 7.17

A selection of the models developed during the gradient descent process for the grass growth dataset from Table 7.9^[367].

Basis functions can also be used to train logistic regression models for categorical prediction problems that involve non-linear relationships. Table 7.10^[370] shows a dataset, the EEG dataset, based on a neurological experiment designed to capture how neural responses change when experiment participants view *positive* images (e.g., a picture of a smiling baby) and *negative* images (e.g., a picture of rotting food). In an experiment performed to capture this data, participants were shown a series of different images, and their neural responses were measured using **electroencephalography (EEG)**. In particular, the values of the commonly used P20 and P45 potentials were measured while a participant viewed each image. These are the descriptive features in this dataset, and the target feature, TYPE, indicates whether the subject was viewing a positive or a negative image. If a model could be trained to classify brain activity as being associated with positive images or negative images, doctors could use this model to help in assessing the brain function of people who have suffered severe brain injuries and are non-communicative.¹⁶ Figure 7.18^[370] shows a scatter plot of this dataset, from which it is clear that the decision boundary between the two different types of images is not linear—i.e., the two types of images are not **linearly separable**.

Table 7.10

A dataset showing participants’ responses to viewing *positive* and *negative* images measured on the EEG P20 and P45 potentials.

ID	P20	P45	TYPE	ID	P20	P45	TYPE
1	0.4497	0.4499	negative	26	0.0656	0.2244	positive
2	0.8964	0.9006	negative	27	0.6336	0.2312	positive
3	0.6952	0.3760	negative	28	0.4453	0.4052	positive
4	0.1769	0.7050	negative	29	0.9998	0.8493	positive
5	0.6904	0.4505	negative	30	0.9027	0.6080	positive
6	0.7794	0.9190	negative	31	0.3319	0.1473	positive

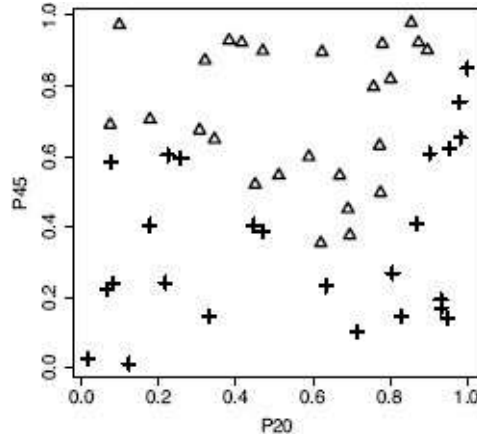


Figure 7.18

A scatter plot of the P20 and P45 features from the EEG dataset. Instances representing *positive* images are shown as crosses, and those representing *negative* images as triangles.

The non-linear decision boundary that is just about perceivable in Figure 7.18^[370] can be represented using a third-order polynomial in the two descriptive features, P20 and P45. The simple regression model we trained previously cannot cope with a non-linear decision boundary like the one seen in Figure 7.18^[370]. We can, however, rewrite the logistic regression equation from Equation (7.26)^[357] to use basis functions as follows:

$$\mathbb{M}_{\mathbf{w}}(\mathbf{d}) = \frac{1}{1 + e^{-\left(\sum_{j=0}^b \mathbf{w}[j] \phi_j(\mathbf{d})\right)}} \quad (7.35)$$

Using this representation with the following set of basis functions will give the learning process the flexibility to find the non-linear decision boundary required to successfully separate the different types of images in the EEG dataset:¹⁷

$$\begin{aligned} \phi_0(\langle \text{P20}, \text{P45} \rangle) &= 1 & \phi_4(\langle \text{P20}, \text{P45} \rangle) &= \text{P45}^2 \\ \phi_1(\langle \text{P20}, \text{P45} \rangle) &= \text{P20} & \phi_5(\langle \text{P20}, \text{P45} \rangle) &= \text{P20}^3 \\ \phi_2(\langle \text{P20}, \text{P45} \rangle) &= \text{P45} & \phi_6(\langle \text{P20}, \text{P45} \rangle) &= \text{P45}^3 \\ \phi_3(\langle \text{P20}, \text{P45} \rangle) &= \text{P20}^2 & \phi_7(\langle \text{P20}, \text{P45} \rangle) &= \text{P20} \times \text{P45} \end{aligned}$$

This model can be trained using gradient descent to find the optimal decision boundary between the two different types of images. Figure 7.19^[372] shows a series of the models built during the gradient descent process. The final model can accurately distinguish between the two different types of image based on the measured P20 and P45 activity. Figure 7.19(f)^[372] shows a 3D plot of the final decision surface. Note that although this decision surface is more complex than the ones we have seen before (for example, Figure 7.12^[358]), the logistic shape is still maintained.

Using basis functions is a simple and effective way in which to capture non-linear relationships within a linear regression model. One way to think about this process is that we change the dataset from two dimensions to a higher dimensional space. There is no limit to the kinds of functions that can be used as basis functions, and as we have seen in the previous example, the basis functions for different descriptive features in a dataset can be quite different. One disadvantage of using basis functions, however, is that the analyst has to design the basis function set that will be used. Although there are some well-known sets of functions—for example, different order polynomial functions—this can be a considerable challenge. Second, as the number of basis functions grows beyond the number of descriptive features, the complexity of our models increases, so the gradient descent process must search through a more complex weight space. Using basis functions is an interesting way to change the inductive bias, in particular the restriction bias, encoded in the gradient descent algorithm for learning regression models. By using basis functions such as those given in the examples in this section, we relax the restriction on the algorithm to consider only linear models and instead allow more complex model types such as the higher order polynomial models seen in these examples.

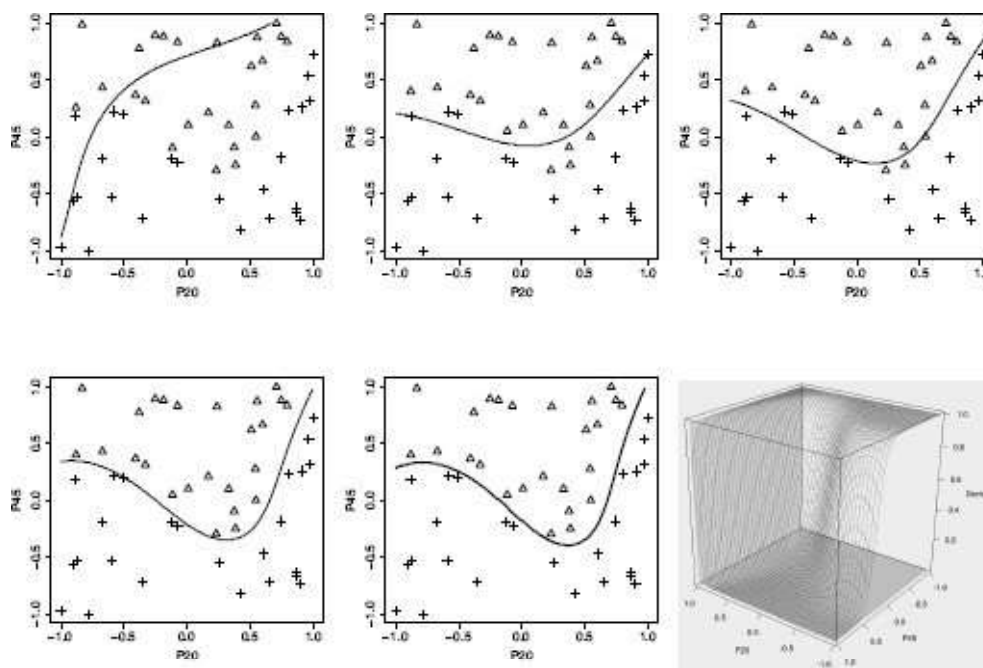


Figure 7.19

A selection of the models developed during the gradient descent process for the EEG dataset from Table 7.10^[370]. The final panel shows the decision surface generated.

7.4.6 Multinomial Logistic Regression

The **multinomial logistic regression**¹⁸ model is an extension that handles categorical target features with more than two levels. A good way to build multinomial logistic regression models is use a set of **one-versus-all** models.¹⁹ If we have r target levels, we create r **one-versus-all** logistic regression models. A **one-versus-all** model distinguishes between one level of the target feature and all the others. Figure 7.20^[373] shows three one-versus-all prediction models for a prediction problem with three target levels (these models are based on the dataset in Table 7.11^[374] that is introduced later in this section).

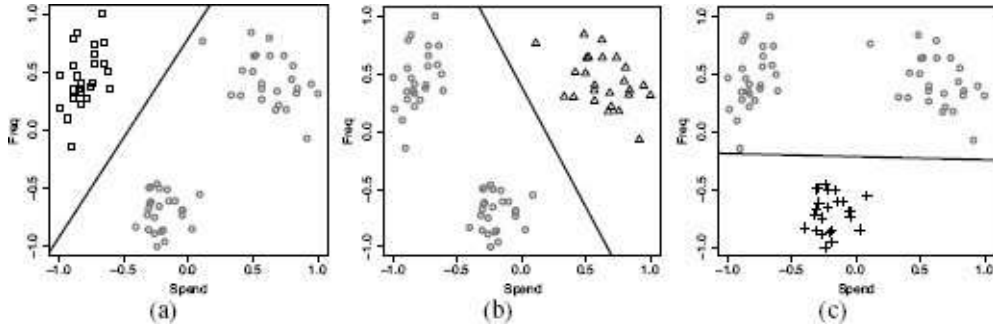


Figure 7.20

An illustration of three different one-versus-all prediction models for the customer type dataset in Table 7.11^[374], with three target levels: *single* (squares), *business* (triangles), and *family* (crosses).

For r target feature levels, we build r separate logistic regression models $\mathbb{M}_{w_1}$ to $\mathbb{M}_{w_r}$:

$$\begin{aligned}\mathbb{M}_{w_1}(\mathbf{d}) &= \text{logistic}(\mathbf{w}_1 \cdot \mathbf{d}) \\ \mathbb{M}_{w_2}(\mathbf{d}) &= \text{logistic}(\mathbf{w}_2 \cdot \mathbf{d}) \\ &\vdots \\ \mathbb{M}_{w_r}(\mathbf{d}) &= \text{logistic}(\mathbf{w}_r \cdot \mathbf{d})\end{aligned}\tag{7.36}$$

where $\mathbb{M}_{w_1}$ to $\mathbb{M}_{w_r}$ are r different one-versus-all logistic regression models, and $\mathbf{w}_1$ to $\mathbf{w}_r$ are r different sets of weights. To combine the outputs of these different models, we normalize their results as follows:

$$\mathbb{M}'_{w_k}(\mathbf{d}) = \frac{\mathbb{M}_{w_k}(\mathbf{d})}{\sum_{l \in \text{levels}(r)} \mathbb{M}_{w_l}(\mathbf{d})}\tag{7.37}$$

Table 7.11

A dataset of customers of a large national retail chain.

ID	SPEND	FREQ	TYPE	ID	SPEND	FREQ	TYPE
1	21.6	5.4	single	28	122.6	6.0	business
2	25.7	7.1	single	29	107.7	5.7	business
3	18.9	5.6	single				
4	25.7	6.8	single				
				47	53.2	2.6	family
				48	52.4	2.0	family
26	107.9	5.8	business	49	46.1	1.4	family
27	92.9	5.5	business	50	65.3	2.2	family

where $\mathbb{M}'_{\mathbf{w}_k}(\mathbf{d})$ is a revised, normalized prediction for the one-versus-all model for the target level k . The denominator in this equation sums the predictions of each of the one-versus-all models for the r levels of the target feature and acts as a normalization term. This ensures that the output of all models sums to 1. The r one-versus-all logistic regression models used are trained in parallel, and the revised model outputs, $\mathbb{M}'_{\mathbf{w}_k}(\mathbf{d})$, are used when calculating the sum of squared errors for each model during the training process. This means that the sum of squared errors function is changed slightly to

$$L_2(\mathbb{M}_{\mathbf{w}_k}, \mathcal{D}) = \frac{1}{2} \sum_{i=1}^n (t_i - \mathbb{M}'_{\mathbf{w}_k}(\mathbf{d}_i[1]))^2 \quad (7.38)$$

The revised predictions are also used when making predictions for query instances. The predicted level for a query, $\mathbf{q}$, is the level associated with the one-versus-all model that outputs the highest result after normalization. We can write this as

$$\mathbb{M}(\mathbf{q}) = \arg \max_{l \in \text{levels}(t)} \mathbb{M}'_{\mathbf{w}_l}(\mathbf{q}) \quad (7.39)$$

Table 7.11^[374] shows a sample from a dataset of mobile customers that includes details of customers' shopping habits with a large national retail chain. Each customer's average weekly spending with the chain, SPEND, and average number of visits per week to the chain, FREQ, are included along with the TYPE of customer: *single*, *business*, or *family*. An extended version of this dataset was used to build a model that can determine the type of a customer based on a few weeks of shopping behavior data. Figure 7.21^[375] shows the training sequence for a multinomial logistic regression model trained using this data (after the data had been range normalized to $[-1, 1]$). There are three target levels, so three one-versus-all models are built. The evolution of the decision boundary for each model is shown.

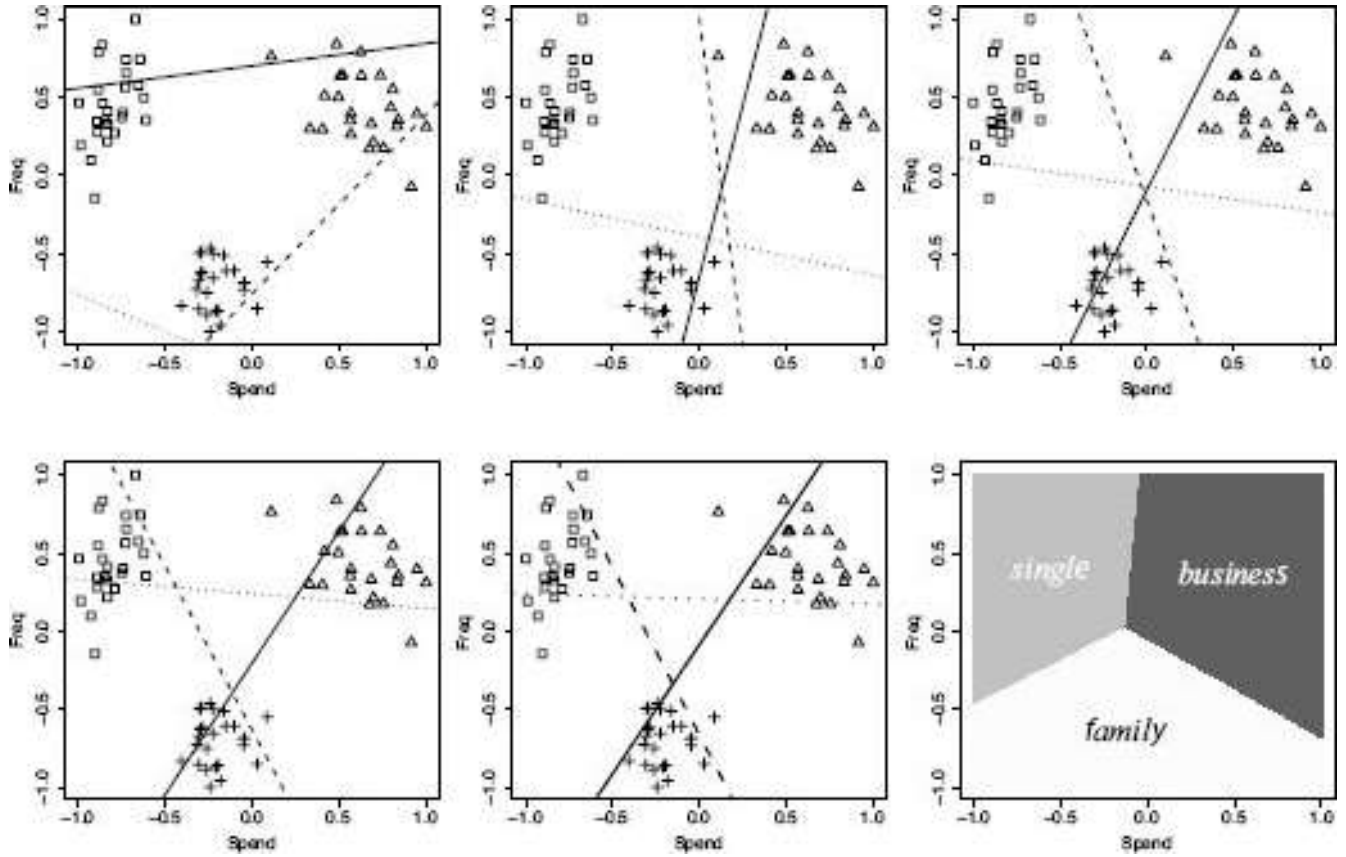


Figure 7.21

A selection of the models developed during the gradient descent process for the customer group dataset from Table 7.11^[374]. Squares represent instances with the *single* target level, triangles the *business* level, and crosses the *family* level. The bottom-right panel illustrates the overall decision boundaries between the three target levels.

The final one-versus-all decision boundaries shown in the bottom-middle panel of Figure 7.21^[375] do not look like the individual one-versus-all decision boundaries shown in Figure 7.20^[373]. The reason for this is that the boundaries shown in Figure 7.20^[373] were trained in isolation, whereas the boundaries shown in Figure 7.21^[375] were trained in parallel and so are interconnected. While it might look like the decision boundary for the *single* target level shown by the solid line does not discriminate between the instances with the *single* target level and those with the other target levels, when used in conjunction with the other two decision boundaries, it does. We can see this in the **decision boundaries** shown in the bottom-right panel of Figure 7.21^[375]. We will use an example to illustrate how a prediction is made using a multinomial regression model.

The parameters of the models learned for the three final decision boundaries in Figure 7.21^[375] are

$$\mathbb{M}_{w_{single}}(\mathbf{q}) = \text{logistic}(0.7993 - 15.9030 \times \text{SPEND} + 9.5974 \times \text{FREQ})$$

$$\mathbb{M}_{w_{family}}(\mathbf{q}) = \text{logistic}(3.6526 + -0.5809 \times \text{SPEND} - 17.5886 \times \text{FREQ})$$

$$\mathbb{M}_{w_{business}}(\mathbf{q}) = \text{logistic}(4.6419 + 14.9401 \times \text{SPEND} - 6.9457 \times \text{FREQ})$$

For a query instance with SPEND = 25.67 and FREQ = 6.12, which are normalized to SPEND

= -0.7279 and FREQ = 0.4789, the predictions of the individual models would be

$$\begin{aligned}\mathbb{M}_{w_{single}}(\mathbf{q}) &= \text{logistic}(0.7993 - 15.9030 \times (-0.7279) + 9.5974 \times 0.4789) \\ &= 0.9999\end{aligned}$$

$$\begin{aligned}\mathbb{M}_{w_{family}}(\mathbf{q}) &= \text{logistic}(3.6526 + -0.5809 \times (-0.7279) - 17.5886 \times 0.4789) \\ &= 0.01278\end{aligned}$$

$$\begin{aligned}\mathbb{M}_{w_{business}}(\mathbf{q}) &= \text{logistic}(4.6419 + 14.9401 \times (-0.7279) - 6.9457 \times 0.4789) \\ &= 0.0518\end{aligned}$$

These predictions would be normalized as follows:

$$\mathbb{M}'_{w_{single}}(\mathbf{q}) = \frac{0.9999}{0.9999 + 0.01278 + 0.0518} = 0.9393$$

$$\mathbb{M}'_{w_{family}}(\mathbf{q}) = \frac{0.01278}{0.9999 + 0.01278 + 0.0518} = 0.0120$$

$$\mathbb{M}'_{w_{business}}(\mathbf{q}) = \frac{0.0518}{0.9999 + 0.01278 + 0.0518} = 0.0487$$

This means the overall prediction for the query instance is *single*, as this gets the highest normalized score.

7.4.7 Support Vector Machines

Support vector machines (SVM) are another approach to predictive modeling that is based on error-based learning. Figure 7.22(a)^[377] shows a scatter plot of a reduced version of the generators dataset (shown in Table 7.6^[354]) with a decision boundary drawn across it. The instance nearest the decision boundary, based on perpendicular distance, is highlighted. This distance from the decision boundary to the nearest training instance is known as the **margin**. The dashed lines on either side of the decision boundary show the extent of the margin, and we refer to these as the **margin extents**.

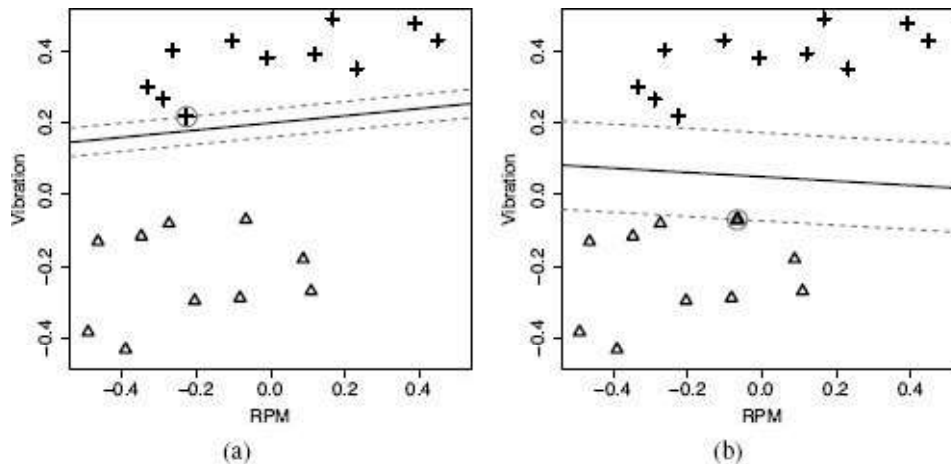


Figure 7.22

A small sample of the generators dataset with two features, RPM and VIBRATION, and two target levels, *good* (shown as crosses) and *faulty* (shown as triangles): (a) a decision boundary with a very small margin; (b) a decision boundary with a much larger margin. In both cases, the instances along the margins are highlighted.

Figure 7.22(b)^[377] shows a similar diagram but with a different decision boundary, which has a much larger margin. The intuition behind support vector machines is that this second decision boundary should distinguish between the two target levels much more reliably than the first. Training a support vector machine involves searching for the decision boundary, or **separating hyperplane**,²⁰ that leads to the maximum margin as this will best separate the levels of the target feature. Although the goal of finding the best decision boundary is the same for algorithms that build support vector machines as it is for logistic regression models, the inductive bias encoded in the algorithms to select this boundary is different, which leads to different decision boundaries being found.

The instances in a training dataset that fall along the margin extents, and so define the margins, are known as the **support vectors**. These are the most important instances in the dataset because they define the decision boundary. There will always be at least one support vector for each level of the target feature, but there is no limit to how many support vectors there can be in total.

We define the separating hyperplane in the same way that we did at the beginning of the discussion of logistic regression: